

UNIVERSIDADE DE SÃO PAULO

Instituto de Ciências Matemáticas e de Computação

Applying Neuroevolution in *Physics Informed Neural Networks* (PINNs)

Gabriel Milanez Cavalheri

Monograph - MBA in Artificial Intelligence and Big Data

SERVIÇO DE PÓS-GRADUAÇÃO DO ICMC-USP

Data de Depósito:

Assinatura: _____

Gabriel Milanez Cavalheri

Applying Neuroevolution in *Physics Informed Neural Networks* (PINNs)

Monograph presented to the Departamento de Ciências de Computação do Instituto de Ciências Matemáticas e de Computação, Universidade de São Paulo - ICMC/USP, as part of the requirements for obtaining the title of Specialist in Artificial Intelligence and Big Data.

Concentration area: Artificial Intelligence and Big Data

Advisor: Prof. Dr. Rodrigo Colnago Contreras

Revised version – English edition

São Carlos

2026

Gabriel Milanez Cavalheri

Aplicando Neuroevolução em *Physics Informed Neural Networks* (PINNs)

Monografia apresentada ao Departamento de Ciências de Computação do Instituto de Ciências Matemáticas e de Computação, Universidade de São Paulo - ICMC/USP, como parte dos requisitos para obtenção do título de Especialista em Inteligência Artificial e Big Data.

Área de concentração: Inteligência Artificial e Big Data

Orientador: Prof. Dr. Rodrigo Colnago Contreras

Versão revisada – edição em inglês

São Carlos

2026

This work is dedicated to my partner and future wife, Elaine, and to my family, especially my parents, Leandro and Patricia, for always supporting me through every moment and difficulty; without them, this work would not exist. It is also dedicated to my advisor, Rodrigo, who guided me into the interesting and current field of PINNs.

ACKNOWLEDGEMENTS

To my advisor, Prof. Dr. Rodrigo Colnago Contreras, for his guidance, his trust, and the discussions that introduced me to *Physics Informed Neural Networks* and to neuroevolution, and that made this work possible.

To my partner and future wife, Elaine, for her support, patience, and encouragement throughout the course.

To my parents, Leandro and Patricia, and to my whole family, for always supporting me through every moment and difficulty.

To the professors and classmates of the MBA in Artificial Intelligence and Big Data at the Institute of Mathematics and Computer Sciences of the University of São Paulo (ICMC-USP), for the knowledge shared throughout the course.

*“An author in his book must be like God in the universe, present everywhere and visible
nowhere.”*

Gustave Flaubert

ABSTRACT

Cavalheri, G. M. **Applying Neuroevolution in *Physics Informed Neural Networks* (PINNs)**. 2026. 70 p. Monograph (MBA in Artificial Intelligence and Big Data) - Instituto de Ciências Matemáticas e de Computação, Universidade de São Paulo, São Carlos, 2026.

The architecture of a neural network, i.e., its number of hidden layers and of neurons per layer, directly affects its accuracy and is usually chosen through costly trial and error. In Physics Informed Neural Networks (PINNs), every trial also requires solving the governing differential equation. This work investigates whether a lightweight neuroevolution procedure can select better PINN architectures. A single-generation, mutation-only Genetic Algorithm evolves the number of hidden layers and of neurons per layer, while the weights are trained with the Adam optimizer; the candidates are ranked through short partial trainings, so the search costs about 1.5 full trainings. First, the pendulum problem shows that, with only ten observations, a neural network trained only on the mean squared error of the data fails to extrapolate the dynamics, whereas the PINN recovers it. Then, the method is evaluated on three classical problems with exact solutions, the simple pendulum, heat transfer in a bar, and the vibrating string, in 35 reproducible runs per problem, against a randomly drawn architecture and against random search with the same epoch budget, using the relative L^2 error against the exact solution. The architecture of the Genetic Algorithm had a lower error than the random architecture in 23, 21, and 26 of the 35 runs, with median error ratios of 4.6, 1.5, and 1.8; the improvement was statistically significant for the pendulum and the vibrating string (Wilcoxon test with Holm correction, $p = 0.027$ and 0.003), but not for heat transfer, which almost any architecture solved. However, the Genetic Algorithm did not outperform random search with the same budget in any problem, which indicates that the benefit comes from evaluating several candidates with a reliable proxy, the partial-training loss, whose Spearman correlation with the error ranged from 0.79 to 0.99, rather than from the evolutionary operators.

Keywords: Neuroevolution. Genetic Algorithms. PINN. Neural Networks. Neural Architecture Search. Pendulum. Wave Equation. Heat Equation.

RESUMO

Cavalheri, G. M. **Aplicando Neuroevolução em *Physics Informed Neural Networks* (PINNs)**. 2026. 70 p. Monografia (MBA em Inteligência Artificial e Big Data) - Instituto de Ciências Matemáticas e de Computação, Universidade de São Paulo, São Carlos, 2026.

A arquitetura de uma rede neural, isto é, o número de camadas ocultas e de neurônios por camada, afeta diretamente sua precisão e costuma ser escolhida por tentativa e erro, de forma custosa. Nas *Physics Informed Neural Networks* (PINNs), cada tentativa exige ainda resolver a equação diferencial que rege o problema. Este trabalho investiga se um procedimento leve de neuroevolução é capaz de selecionar arquiteturas de PINN melhores. Um Algoritmo Genético de uma única geração, baseado apenas em mutação, evolui o número de camadas ocultas e de neurônios por camada, enquanto os pesos são treinados com o otimizador Adam; as candidatas são ranqueadas por treinamentos parciais curtos, de modo que a busca custa cerca de 1,5 treinamento completo. Inicialmente, mostra-se no problema do pêndulo que, com apenas dez observações, uma rede neural treinada somente com o erro quadrático médio dos dados não extrapola a dinâmica do sistema, enquanto a PINN a recupera. Em seguida, o método é avaliado em três problemas clássicos com solução exata, o pêndulo simples, a transferência de calor em uma barra e a corda oscilante, em 35 rodadas reprodutíveis por problema, contra uma arquitetura sorteada e contra uma busca aleatória com o mesmo orçamento de épocas, usando o erro L^2 relativo em relação à solução exata. A arquitetura do Algoritmo Genético teve erro menor do que a arquitetura aleatória em 23, 21 e 26 das 35 rodadas, com razões medianas de erro de 4,6, 1,5 e 1,8; a melhora foi estatisticamente significativa no pêndulo e na corda oscilante (teste de Wilcoxon com correção de Holm, $p = 0,027$ e $0,003$), mas não na transferência de calor, que quase qualquer arquitetura resolveu. O Algoritmo Genético, porém, não superou a busca aleatória com o mesmo orçamento em nenhum dos problemas, o que indica que o benefício vem de avaliar várias candidatas com um indicador confiável, o custo do treinamento parcial, cuja correlação de Spearman com o erro variou de 0,79 a 0,99, e não dos operadores evolutivos.

Palavras-chave: Neuroevolução. Algoritmos Genéticos. PINN. Redes Neurais. Busca de Arquitetura Neural. Pêndulo. Equação da Onda. Equação do Calor.

LIST OF FIGURES

Figure 1 – Flowchart of the neuroevolution of the PINN architecture.	39
Figure 2 – Exact solution of the pendulum, training data, and collocation points of the physics loss.	54
Figure 3 – Values predicted by the standard neural network after 200 and 1,000 training epochs.	55
Figure 4 – Values predicted by the PINN after 6,000, 12,000, 18,000, and 20,000 training epochs.	56
Figure 5 – Loss of the standard neural network and terms of the loss of the PINN, as a function of the training epochs.	57
Figure 6 – Exact solution of the heat transfer problem used in this work: tempera- ture profiles at five instants and map of $u(x, t)$	59
Figure 7 – Exact solution of the vibrating string problem used in this work: dis- placement profiles at five instants and map of $u(x, t)$	60
Figure 8 – Relative L^2 error of the final network of each arm in the 35 runs of each problem, on a logarithmic scale. The boxes show the quartiles, and each point is one run.	61
Figure 9 – PINN obtained with the GA architecture in the median-error run of each problem. Top: pendulum, with the ten observations. Middle and bottom: exact solution, PINN prediction, and absolute error for heat transfer and for the vibrating string.	62
Figure 10 – Total loss along the full training of the three arms in the median-error GA run of each problem. The thin lines show the loss at every epoch, and the thick lines its rolling median over 200 epochs. The legends give the architectures as hidden layers $\times$ neurons per layer.	65

LIST OF TABLES

Table 1	– Configuration of the experiments, identical for the three problems. . . .	52
Table 2	– The 5 architectures selected in the screening of run 5 of the pendulum, with their <i>fitness</i> after E_{fit} epochs, the offspring generated by mutation, and the <i>fitness</i> of both after E_{term} epochs. Architectures are given as (hidden layers, neurons per layer).	57
Table 3	– Architectures of the three arms in run 5 of the pendulum and relative L^2 error of the PINN after full training.	58
Table 4	– Median relative L^2 error and median number of trainable parameters of the final network of each arm (35 runs per problem), and Spearman correlation ρ_S between the <i>fitness</i> and the relative L^2 error of the 350 candidates screened by the GA in each problem.	61
Table 5	– Paired comparisons of the GA with each reference over the 35 runs of each problem, on the relative L^2 error. Wins: runs in which the GA obtained the lower error, with the Wilson 95% interval of the win rate; p_B : one-sided binomial test; p_W : one-sided Wilcoxon signed-rank test; both Holm-adjusted over the six comparisons. Median ratio: median of the reference error divided by the GA error, with a bootstrap 95% interval (values above 1 favor the GA). Wins (loss): wins on the secondary metric.	63

LIST OF ABBREVIATIONS AND ACRONYMS

CNN	Convolutional Neural Network
EC	Evolutionary Computation
GA	Genetic Algorithm
GPU	Graphics Processing Unit
GRU	Gated Recurrent Unit
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
MLP	Multilayer Perceptron
MSE	Mean Squared Error
NEAT	NeuroEvolution of Augmenting Topologies
ODE	Ordinary Differential Equation
PDE	Partial Differential Equation
PINN	Physics Informed Neural Network
PSO	Particle Swarm Optimization
ReLU	Rectified Linear Unit
SGD	Stochastic Gradient Descent

CONTENTS

1	INTRODUCTION	23
2	MATHEMATICAL BACKGROUND	27
2.1	Neural Networks	27
2.2	PINN	29
2.3	Neuroevolution	31
2.3.1	Evolutionary Computation and Genetic Algorithms	31
2.3.2	Neuroevolution	33
2.3.3	Individual Representation	34
2.3.4	Fitness Function	35
2.3.5	Evolutionary Operators	36
2.3.6	Neuroevolution Algorithm	37
2.3.7	Neuroevolution Methods in the Literature	38
2.3.8	Neuroevolution and Gradient-Based Training	40
2.3.9	Complexity and Limitations	41
2.3.10	Neuroevolution Applied to This Work	42
3	RELATED WORK	45
3.1	Physics Informed Neural Networks	45
3.2	Neuroevolution in Neural Networks and PINNs	45
4	METHODOLOGY	47
4.1	Overview of the Method	47
4.2	Genetic Algorithm	47
4.3	Validation Strategy	47
4.3.1	Comparison Arms	47
4.3.2	Metrics	48
4.3.3	Statistical Analysis	48
4.4	Reproducibility	49
5	EXPERIMENTAL EVALUATION	51
5.1	Computational Setup	51
5.2	Neuroevolution Configuration	51
5.3	Pendulum	52
5.3.1	Standard Neural Network and PINN	53
5.3.2	Example of a Neuroevolution Run	55
5.4	Heat Transfer in a One-Dimensional Bar	58

5.5 **Vibrating String 59**

5.6 **Neuroevolution Results 60**

5.6.1 GA versus Random Architecture 63

5.6.2 GA versus Random Search with the Same Budget 63

5.6.3 Partial Training as a Proxy 64

5.6.4 Selected Architectures and Computational Cost 64

5.7 **Discussion 64**

6 **CONCLUSIONS 67**

REFERENCES 69

1 INTRODUCTION

Artificial neural networks (Géron, 2019) are very powerful statistical tools for regression and classification problems when data about the problem are available. In physical problems related to the heat equation, such as heat transfer in a bar, to the wave equation, such as the vibrating string, and to oscillations, such as the pendulum (Thornton; Marion, 2004), it is very common to take many measurements during the experimental phase, so that the data can later be analyzed and extrapolated. However, it is also very common that, during data collection, several variables interfere with a systematic and standardized acquisition. This results in missing data and in different sampling intervals.

Neural networks usually rely on a single loss function, whose goal is to minimize the error between the expected value and the value predicted by the model during training and validation. This loss function tends to work well for general problems with plenty of well-distributed data.

This work addresses three scenarios that are extremely unfavorable for conventional neural networks: physical problems that evolve in time and for which only a few initial data points are available (Brigato; Iocchi, 2021). In this scenario, regardless of the training time or of the size of the architecture, the predicted values will not generalize or extrapolate to the missing data.

The physics-informed neural network architecture, *Physics Informed Neural Networks* (PINN) (Raissi; Perdikaris; Karniadakis, 2019), can generalize, extrapolate, and adapt to the few data already collected, because physical information is added beforehand to the loss function of the neural network.

The choice of the architecture of an artificial neural network, i.e., the number of layers and of neurons in each layer, is one of the main challenges in the development of these models, since it directly affects their performance and generalization ability. This choice is commonly based on exhaustive experimental analyses, which can be extremely costly in time and computational resources. For instance, the size of the flagship Llama 3 model (Grattafiori *et al.*, 2024) was chosen from scaling-law experiments in which models from 40 million to 16 billion parameters were pre-trained, and the pre-training of the final 405-billion-parameter model alone used up to 16 thousand GPUs, with a 54-day snapshot of that training reported by the authors. This scenario raises a central question: how can one ensure that the best architecture was chosen without fully training the model for every configuration tested?

Neuroevolution (Akut; Kulkarni, 2019) is the optimization of neural network structures, which can identify a good architecture without fully training every artificial

neural network. It relies on metaheuristics or evolutionary strategies, for example, Genetic Algorithms (Holland, 1992), to select the best attributes during the initial and intermediate stages of training. This concept is based on the selection of desired characteristics of individuals (Stanley *et al.*, 2019), which in this work are the architectures of the PINNs, aiming at a better result than simply selecting an architecture at random.

The objective of this work is to apply neuroevolution to the choice of the architecture of PINNs and to assess, in a reproducible way, whether it selects better architectures than a random choice and than a random search with the same computational budget. To this end, we propose a single-generation, mutation-only Genetic Algorithm that evolves the number of hidden layers and of neurons per layer of the PINN, while the weights are still trained by gradient descent, and that evaluates each candidate through partial training, so that the complete search costs about 1.5 full trainings. The method is evaluated on three classical physical problems with exact solutions, the simple pendulum, heat transfer in a bar, and the vibrating string, in 35 reproducible runs per problem, using the error of the PINN with respect to the exact solution as the primary metric. We seek to answer three questions: (i) is the architecture chosen by the Genetic Algorithm more accurate than a randomly chosen architecture? (ii) do the evolutionary operators add value over simply evaluating the same number of randomly drawn candidates with the same epoch budget? and (iii) is the loss after a partial training a good indicator of the final quality of an architecture?

About this revised version. After the original version of this monograph was submitted, we found that the networks of the heat transfer and vibrating string experiments used the ReLU activation function. Since a ReLU network is piecewise linear in its inputs, its second derivatives vanish, so the terms $\partial^2 u / \partial x^2$ and $\partial^2 u / \partial t^2$ of the equations of these problems were not represented by the PINN. In this version, all experiments were redesigned and rerun: all networks use the hyperbolic tangent, the problems were formulated in dimensionless form with known exact solutions, the error with respect to these solutions became the primary metric, all random decisions use fixed seeds, and random search with the same budget was added as a second reference. The results presented in Chapters 5 and 6 replace those of the original version, and the code and complete results are publicly available¹. This English edition is a translation of the revised version, originally written in Portuguese.

This work is organized as follows: Chapter 2 presents the necessary mathematical background on neural networks and PINNs; Chapter 3 discusses related work, reviewing the literature on PINNs and on the application of neuroevolution to the optimization of neural networks and PINNs; Chapter 4 describes the methodology, including the validation strategy and the statistical analysis; Chapter 5 presents the experimental setup and

¹ <https://github.com/GMCavaleri/MBA-PINN-Neuroevolution>

discusses the results obtained for the three physical problems; finally, Chapter 6 presents the conclusions of the work.

2 MATHEMATICAL BACKGROUND

This chapter presents the theoretical concepts needed to understand this work. First, Section 2.1 reviews the fundamentals of artificial neural networks, based mainly on the book by Géron (2019). Next, Section 2.2 presents the concept of PINN, introduced by Raissi, Perdikaris and Karniadakis (2019) and later consolidated by the *survey* of Karniadakis *et al.* (2021) and by the review of Cuomo *et al.* (2022), which map the state of the art and the main open challenges of this approach. Finally, Section 2.3 presents the fundamentals of evolutionary computation and Genetic Algorithms (Holland, 1992; Goldberg, 1989), the concept of neuroevolution and its different forms of application to neural networks (Yao, 1999), and the formalization of the neuroevolution method used in this work to optimize the architecture of PINNs. The combination of neuroevolution and PINNs is still little explored in the literature, and the *survey* of Wong *et al.* (2026) is one of the few references that specifically discusses evolutionary strategies applied to the optimization of PINNs, which reinforces the relevance of the topic addressed in this monograph.

2.1 Neural Networks

A fully connected neural network, also known as a *Multilayer Perceptron* (MLP) (Géron, 2019), is composed of several neurons, called *Perceptrons*, organized in interconnected layers. An MLP has three types of layers: input, hidden, and output. In general, the output of each layer l can be written as

$$a^{(l)} = f \left(W^{(l)} a^{(l-1)} + b^{(l)} \right), \quad (2.1)$$

where $a^{(l-1)}$ is the output vector of the previous layer (with $a^{(0)} = x$ being the input vector of the network), $W^{(l)}$ and $b^{(l)}$ are, respectively, the weight matrix and the *bias* vector of layer l , and f is the activation function.

The input layer must have one neuron for each input attribute of the data. The hidden layers vary widely depending on the architecture of the network and connect the input layer to the output layer; their number can range from one to many layers, and their main role, besides connecting the data that enter and leave the network, is to learn and store knowledge through successive interactions with the training data. The output layer must have one neuron for each target attribute, i.e., for each value to be predicted (Géron, 2019).

Learning is possible because each neuron has free parameters, also known as synaptic weights (*weights*), which are not fixed and can change when exposed to new data,

also called input signals x_n (*inputs*). The input signals are multiplied by their respective synaptic weights w_n and summed in a linear function z (*linear function*), together with a constant b (*bias*) (Géron, 2019):

$$z = w^T x + b. \quad (2.2)$$

The value of the linear function is then passed to another function, called the activation function (*step function*), which transforms the sum into an output value $h_{w,b}(x)$ (*output*):

$$h_{w,b}(x) = \text{step}(z) = \text{step}(w^T x + b). \quad (2.3)$$

One of the most used metrics to evaluate the quality of the learning of a neural network is the loss function (*loss function*), also called objective function, which quantifies the error between the values predicted by the network and the expected values for each sample of the dataset. The goal of training is to find the set of parameters (weights and *biases*) that minimizes this function.

There are many loss functions in the literature, each more suitable for a given type of problem. For regression problems, such as those addressed in this work, one of the most used is the Mean Squared Error (MSE), defined as

$$MSE = \frac{1}{N} \sum_{i=1}^N (\bar{x}_i - x_i)^2, \quad (2.4)$$

where N is the total number of samples considered, and $\bar{x}_i$ and x_i are, respectively, the predicted and the expected values for the i -th sample. Besides the MSE, other loss functions are also widely used, such as the Mean Absolute Error (MAE), which is more robust to *outliers*, and the Cross-Entropy, commonly used in classification problems.

The loss function is minimized by optimization algorithms based on gradient descent, which iteratively adjust the parameters of the network in the direction opposite to the gradient of the loss function with respect to these parameters. This principle of learning by error correction goes back to the Adaline (*Adaptive Linear Neuron*), proposed by Widrow and Hoff (1960), one of the first artificial neuron models trained with the Delta rule, which adjusts the weights in proportion to the error between the predicted and the desired outputs:

$$e_k(n) = \bar{x}_k(n) - x_k(n), \quad (2.5)$$

where n is the index of the sample in the training set, and k is the index of the corresponding neuron of the output layer. This same principle of error-based adjustment underlies the

backpropagation algorithm, currently used to train multilayer neural networks by gradient descent.

The network learns through the following procedure: the input signals propagate through the whole network, the free parameters of the neurons are adjusted, and output signals are produced in the domain of the target attributes. At each iteration of this process, the value of the loss function tends to decrease and approach zero. As the loss approaches zero, the predicted values tend to interpolate the training data; however, if the model cannot keep the loss low when exposed to test data, *overfitting* occurs, and the model fails to generalize to new data.

The choice of the activation function f (Equation 2.1) also influences what the network can represent. Two of the most used are the hyperbolic tangent, $f(z) = \tanh(z)$, and the rectified linear unit (ReLU), $f(z) = \max(0, z)$ (Géron, 2019). ReLU is cheap to compute and mitigates vanishing gradients in deep networks, which made it the default choice in many applications. However, because it is piecewise linear, a network composed only of ReLU layers is also a piecewise linear function of its inputs: its first derivative is piecewise constant, and its second derivative vanishes almost everywhere. The hyperbolic tangent, on the other hand, is smooth and has derivatives of all orders. This difference is irrelevant when the network is trained only on data, but it is decisive for PINNs, presented next, whose loss function involves derivatives of the network with respect to its inputs.

2.2 PINN

Many physical phenomena are described by differential equations, which relate an unknown function to its derivatives. When this function depends on a single independent variable, the equation is called an Ordinary Differential Equation (ODE); when it depends on two or more independent variables, such as time and one or more spatial coordinates, and involves partial derivatives with respect to these variables, it is called a Partial Differential Equation (PDE).

In general, following the formulation of Raissi, Perdikaris and Karniadakis (2019), the physical problem can be stated as follows: let $u(x)$ be the unknown solution of interest, defined on a domain $\Omega \subset R^D$, with $x = (x_1, \dots, x_D)$ representing the independent variables of the problem, for example, position and time. We seek u such that

$$\mathcal{N}[u](x) = 0, \quad x \in \Omega, \quad (2.6)$$

subject to the boundary conditions

$$\mathcal{B}[u](x) = 0, \quad x \in \partial\Omega, \quad (2.7)$$

where $\mathcal{N}$ is a linear or nonlinear differential operator that may involve derivatives of u of any order, Ω is the domain of the problem, and $\partial\Omega$ is its boundary, where the boundary conditions are imposed.

Traditionally, a neural network trained to approximate $u(x)$ uses only a loss function based on observed data, such as the MSE (Equation 2.4), which penalizes the difference between the values predicted by the network and the measured or known values of u at a finite set of points. However, this approach does not ensure that the learned solution satisfies Equation 2.6 away from the training points, which compromises its ability to generalize and extrapolate, especially when few data are available.

The PINN approach, proposed by Raissi, Perdikaris and Karniadakis (2019), solves this problem by incorporating the differential operator $\mathcal{N}$ itself into the loss function of the network. In this approach, the solution $u(x)$ is approximated by a neural network $\hat{u}(x; \theta)$, parameterized by the weights θ of the network. Besides the observed data, a set of collocation points $\{x_c\}_{c=1}^{N_c} \subset \Omega$ is sampled in the domain of the problem, at which the residual of the differential equation is computed:

$$r(x) = \mathcal{N}[\hat{u}](x; \theta). \quad (2.8)$$

The residual $r(x)$ measures how much the approximate solution $\hat{u}$ fails to satisfy Equation 2.6 at point x : the closer it is to zero, the more faithfully $\hat{u}$ obeys the differential equation at that point, and an exactly zero residual means that the network satisfies the physics of the problem exactly at x . Since $\mathcal{N}$ may involve derivatives of $\hat{u}$ of any order with respect to its inputs, computing $r(x)$ requires these derivatives, which are obtained by automatic differentiation, natively available in deep learning libraries such as PyTorch.

The residual at the collocation points is then aggregated into a second loss function, called the physics loss, defined as the mean squared residual over these points:

$$\mathcal{L}_{\text{phys}}(\theta) = \frac{1}{N_c} \sum_{c=1}^{N_c} r(x_c)^2. \quad (2.9)$$

Unlike the data loss, which penalizes the deviation of the network from observed values of u , the physics loss does not depend on any measured data: it only penalizes how much the learned solution fails to satisfy the differential equation at the sampled collocation points, acting as a physics-based regularization. The number of collocation points N_c and their distribution in the domain Ω depend on the physical problem and are detailed in Chapter 5 for each of the three cases studied in this work.

The total loss of the PINN is the sum of the data loss ($\mathcal{L}_{\text{data}}$, usually the MSE) and the physics loss ($\mathcal{L}_{\text{phys}}$, Equation 2.9):

$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}_{\text{data}}(\theta) + \mathcal{L}_{\text{phys}}(\theta). \quad (2.10)$$

The initial and boundary conditions of the problem (Equation 2.7) are imposed in the same way, as additional mean squared error terms evaluated at points sampled at the initial instant and on the boundaries of the domain; in this case, they are said to be imposed weakly (*soft constraints*), since they are only penalized, not guaranteed by the construction of the network. When the terms have very different scales, the physics loss can also be multiplied by a weight $\lambda > 0$ to balance its contribution with that of the other terms.

Since the residual $r(x)$ involves derivatives of $\hat{u}$ with respect to the inputs, the network must be smooth enough to represent them. In particular, if the equation contains second derivatives, such as $\partial^2 u / \partial x^2$ or $\partial^2 u / \partial t^2$, a network with ReLU activations does not represent them, because its second derivatives vanish almost everywhere (Section 2.1): the corresponding term simply disappears from the residual, and the PINN ends up solving a different equation from the intended one. For this reason, all networks in this work use the hyperbolic tangent as activation function.

By minimizing this combined loss, the network is forced to find a solution that not only fits the observed data but also respects the physics described by the differential equation over the whole domain, which allows the model to generalize and extrapolate correctly to regions without data. This property makes PINNs particularly suitable for problems with few initial data, such as the three physical problems addressed in this work, whose specific formulations and the particular form of the operator $\mathcal{N}$ in each case are presented in Chapter 5.

2.3 Neuroevolution

2.3.1 Evolutionary Computation and Genetic Algorithms

Evolutionary Computation (EC) is a family of stochastic, population-based search and optimization techniques inspired by the mechanisms of natural evolution: natural selection, reproduction, and genetic variation (Eiben; Smith, 2015). This umbrella includes different paradigms, developed relatively independently during the second half of the twentieth century, such as Genetic Algorithms, Evolution Strategies, Evolutionary Programming, and Genetic Programming. Despite their differences in representation and operators, they all share the same conceptual structure: a set of candidate solutions is evaluated for quality, and new solutions are iteratively generated from the best solutions of the previous generation through variation and selection operators (Eiben; Smith, 2015).

This work focuses specifically on Genetic Algorithms (GA), formalized by Holland (1992) and systematized by Goldberg (1989) as a search method that represents each

candidate solution as a string of symbols, analogous to biological chromosomes, on which operators inspired by Mendelian genetics act. Before describing the GA procedure, it is important to define the terminology precisely, since the imprecise use of these terms is a common source of confusion in the applied literature:

- Individual: a complete candidate solution to the problem.
- Chromosome (or genotype): the encoded representation of an individual, i.e., the data structure on which the evolutionary operators act directly.
- Gene: each elementary component or position of the chromosome, corresponding to an encodable aspect of the solution.
- Allele: the specific value taken by a gene in a particular individual.
- Population: the set of individuals kept and evolved simultaneously in the same generation.
- *Fitness*: a numerical measure of the quality of an individual with respect to the problem, used to guide selection.

An individual corresponds to the complete solution and should not be confused with a gene, which represents only an elementary component of that solution. This distinction, revisited in Section 2.3.3 for the specific case of neural networks, is essential to understand how different aspects of a solution can be encoded and evolved independently.

In general, the GA procedure can be divided into six steps (Holland, 1992; Goldberg, 1989): population initialization, *fitness* evaluation, selection, crossover and mutation, replacement, and stopping criterion. In the initialization, an initial population of individuals is generated at random. In the *fitness* evaluation, each individual undergoes problem-specific tests that measure its quality. In the selection, the individuals with the best *fitness* are chosen to produce offspring, while the others tend to be discarded. In crossover and mutation, new individuals are generated from the selected ones by combining (crossover) and/or randomly altering (mutation) their genetic material, in the expectation that the offspring outperform their ancestors. In the replacement, the offspring take the place of the worst individuals, keeping the population size constant. Finally, the stopping criterion determines when the evolutionary process ends, usually after a fixed number of generations or when the *fitness* of the population stabilizes.

This generic structure, however, is not enough to describe how a GA can be used to optimize neural networks, which is the central question of this work and is discussed from Section 2.3.2 onward.

2.3.2 Neuroevolution

Neuroevolution is the application of evolutionary computation techniques, especially Genetic Algorithms, to the design and/or training of artificial neural networks (Stanley *et al.*, 2019; Yao, 1999). Training a neural network can be seen as an optimization problem: one searches, in the space of possible configurations of the network, for the one that minimizes (or maximizes) a performance measure on a dataset. This is exactly what an evolutionary algorithm solves generically, treating each candidate configuration as an individual and evaluating its quality through a *fitness* function. Unlike gradient descent, a GA does not require the function to be optimized to be differentiable with respect to the evolved parameters, which makes it suitable for discrete or combinatorial search spaces, such as the number of layers or neurons of an architecture (Yao, 1999).

According to the survey by Yao (1999), neuroevolution can act on different aspects of a neural network, alone or in combination:

- Synaptic weights: the individual directly encodes the values of the weights and biases of the network, whose architecture is kept fixed. The *fitness* is obtained by running the resulting network on the data of the problem.
- Architecture (topology): the individual encodes structural characteristics of the network, such as the number of hidden layers and the number of neurons per layer. In this case, the weights are typically obtained by a conventional training algorithm, such as gradient descent, applied to each candidate architecture.
- Hyperparameters: the individual encodes parameters of the training process itself, such as the learning rate, the batch size, or regularization coefficients.
- Activation functions: the individual encodes the choice of the activation function of each layer or neuron.
- Combinations of these elements: different aspects can be evolved simultaneously by the same individual.

A particularly relevant distinction is between evolving only the weights of a fixed architecture and evolving weights and topology simultaneously. In the first case, the search space has a fixed dimensionality equal to the number of parameters of the network; this approach does not need gradients but tends to scale poorly with the number of parameters, becoming uncompetitive with gradient descent in deep networks (Yao, 1999). In the second case, both the structure and the values of the parameters evolve over the generations; the NeuroEvolution of Augmenting Topologies (NEAT) method, proposed by Stanley and Miikkulainen (2002), is the most influential example of this category and is revisited in Section 2.3.7.

This work belongs to a third category, intermediate between the two previous ones: only the architecture of the network, i.e., the number of hidden layers and of neurons per layer, is evolved, and the weights are kept outside the evolutionary process. The weights of each candidate architecture are still obtained by gradient-based training with the Adam optimizer (Kingma; Ba, 2014), as detailed in Chapter 4. This approach, also used by Akut and Kulkarni (2019) for the automatic design of deep learning architectures, avoids the cost of directly evolving a very high-dimensional weight space and concentrates the evolutionary effort exclusively on the structural choice of the network.

2.3.3 Individual Representation

As discussed in Section 2.3.2, how an individual is encoded depends on which aspects of the neural network are being evolved. Two encodings illustrate this point well.

When the goal is to evolve the weights of a fixed architecture, the chromosome is usually a real vector obtained by concatenating all weight matrices $W^{(l)}$ and bias vectors $b^{(l)}$ of the network (Equation 2.1), flattened into a single sequence of numbers. In this case, each gene corresponds to a single weight or bias, and the allele is the real value taken by that parameter.

When the goal is to evolve the architecture of the network, as in this work, the chromosome encodes structural characteristics instead of numerical weight values. An individual A_i can be represented as

$$A_i = (L_i, n_i), \quad (2.11)$$

where L_i is the number of hidden layers of the architecture and n_i is the number of neurons of each hidden layer. All hidden layers of an architecture have the same number of neurons, while the input and output layers are defined by the physical problem, with one neuron for each input and output variable. In this case, each gene corresponds to a structural characteristic of the network: the first gene is the number of hidden layers, and the second is the number of neurons per hidden layer; the allele is the integer value taken by that gene in a particular individual. For example, an architecture with three hidden layers of 32 neurons each is encoded by the chromosome $(3, 32)$; the allele of the gene corresponding to the number of neurons is, in this individual, equal to 32.

Although the values of L_i and n_i may vary between individuals, the chromosome defined by Equation 2.11 has a fixed length, equal to 2, for all individuals. This compact representation reduces the search space to pairs (L_i, n_i) within the bounds defined in Section 5.2, at the cost of not covering architectures with different numbers of neurons in different hidden layers.

2.3.4 Fitness Function

The *fitness* function connects the evolutionary algorithm to the problem to be solved. It assigns to each individual a numerical value that represents its quality, and this value guides the selection: individuals with better *fitness* are more likely to produce offspring, while the others tend to be discarded over the generations.

The concrete definition of the *fitness* depends on the type of problem and on the aspect of the network being evolved (Section 2.3.2). In classification problems, it is common to use the accuracy or the cross-entropy on a validation set. In regression problems, such as those addressed in this work, it is common to use the mean squared error (Equation 2.4) or another error measure between the predicted and the expected values. More generally, when the neural network is used to solve an optimization problem, such as a differential equation, the *fitness* can be defined directly as the value of the objective function of that problem.

This choice also determines whether the algorithm should maximize or minimize the *fitness*. Quality measures, such as accuracy, should be maximized: the higher the value, the better the individual. Error or loss measures, such as the MSE, should be minimized: the lower the value, the better the individual. Both formulations are equivalent, as it suffices to invert the criterion used in the selection step; the choice between them is usually a matter of mathematical convenience or of consistency with the metric already used to train the network.

This work adopts the second formulation. For each candidate architecture A_i (Equation 2.11), the corresponding PINN, with weights θ_i , is trained for a reduced number of epochs E_{fit} , corresponding to a fraction of the total number of training epochs of a PINN. Since the value of the loss function oscillates from one epoch to the next during training with Adam, sometimes with spikes of several orders of magnitude, the *fitness* of A_i is defined as the mean of the total loss of the PINN (Equation 2.10) over the last W epochs of this partial training:

$$f(A_i) = \frac{1}{W} \sum_{e=E_{fit}-W+1}^{E_{fit}} \mathcal{L}_{\text{total}}^{(e)}(\theta_i), \quad (2.12)$$

where $\mathcal{L}_{\text{total}}^{(e)}$ is the value of the total loss at epoch e . The *fitness* uses only the loss of the PINN and never the exact solution of the problem, which would be unknown in a real application. Since lower values of f indicate better performance, the algorithm seeks to minimize the *fitness* of each individual. Partial training, instead of full training, is what makes evaluating multiple candidate architectures affordable: training each of them for the full E_{total} epochs would make the computational cost of the evolutionary search prohibitive, approaching that of an exhaustive search over the space of architectures.

2.3.5 Evolutionary Operators

The evolutionary operators are the mechanisms that drive the population toward better solutions over the generations. In general, four operators are distinguished: selection, crossover (or recombination), mutation, and replacement; elitism, when present, is not a separate operator but a rule applied during selection or replacement.

Selection is the operator that determines which individuals of the current population take part in reproduction. Among the most common strategies in the literature are roulette-wheel selection, in which the probability of an individual being chosen is proportional to its *fitness*; tournament selection, in which small subsets of individuals compete with each other and the one with the best *fitness* wins; and truncation selection, in which only the individuals with the best *fitness* are kept and the rest of the population is discarded (Eiben; Smith, 2015; Goldberg, 1989). This work adopts truncation selection: the initial population P_0 is sorted by increasing *fitness*, and the subset with the K best individuals is selected:

$$P_0^{select} = \{A_{(1)}, \dots, A_{(K)}\}, \quad f(A_{(1)}) \leq f(A_{(2)}) \leq \dots \leq f(A_{(K)}). \quad (2.13)$$

Crossover (or recombination) is the operator that combines the genetic material of two selected individuals to generate one or more offspring, in the expectation that the combination preserves advantageous characteristics of both parents. In a fixed-length chromosome, a common form is one-point crossover: given a cut point k , the offspring inherits the genes before k from one parent and the genes after k from the other. This operator, however, is not used in this work. With the two-gene chromosome defined by Equation 2.11, crossover would merely combine the number of layers of one parent with the number of neurons per layer of the other. This work opts for reproduction based exclusively on mutation, so that each offspring is a small variation of one of the best architectures of the screening. This choice keeps the procedure simple and the search concentrated around the selected candidates, at the cost of a more conservative exploration of the search space, since each offspring derives from a single individual rather than from the combination of two.

Mutation is the operator that introduces variation in the population through random changes in the chromosome of an individual. In a classical GA with a binary chromosome, for example, each gene is flipped with a small fixed probability, called the mutation rate. In this work, mutation follows a different logic: instead of changing each gene with a given probability, a random non-negative increment is applied, with probability 1, to each gene of the selected individual, generating a single offspring per individual. Formally, from P_0^{select} , K offspring are generated by a mutation operator μ applied to each selected architecture:

$$A'_i = \mu(A_i) = (L_i + \Delta L_i, n_i + \Delta n_i), \quad (2.14)$$

where ΔL_i and Δn_i are random non-negative increments drawn from discrete uniform distributions with predefined bounds, applied respectively to the number of hidden layers and to the number of neurons per layer. Thus, the offspring may gain one hidden layer or none, and the number of neurons of all its hidden layers, including the possibly added one, increases by the same amount or stays the same; the concrete values of these bounds are given in Section 5.2. Since the increments are always non-negative, mutation tends to generate architectures equal to or larger than the original ones, never smaller.

Replacement is the operator that defines how the generated offspring join the next population. Common strategies include generational replacement, in which the whole population is replaced by the offspring, and steady-state replacement, in which only a few individuals are replaced at a time. This work adopts elitist replacement: the K individuals selected in P_0^{select} remain unchanged, while the offspring take the place of the K worst individuals of the original population, keeping the population size M constant and resulting in the final population P_1 :

$$P_1 = P_0^{select} \cup \{A'_1, \dots, A'_K\}. \quad (2.15)$$

This replacement strategy already includes a form of elitism: since P_0^{select} contains exactly the K best individuals of P_0 , and these individuals are preserved unchanged in P_1 , the *fitness* of the best individual does not worsen between the initial and the final population. Note that P_0 and P_1 denote, respectively, the initial population and the population resulting from the single generation executed by Algorithm 1 (Section 2.3.6), not a generic generation index g : as discussed below, this method does not iterate over multiple generations.

2.3.6 Neuroevolution Algorithm

Gathering the elements defined in the previous sections (individual representation, *fitness* function, and evolutionary operators), Algorithm 1 describes the neuroevolution procedure used in this work to search for the architecture of the PINN.

Figure 1 illustrates the same procedure as a flowchart, which makes explicit the transition between the initial population P_0 , evaluated with E_{fit} epochs only for screening, and the final population P_1 , re-evaluated with E_{term} epochs before choosing A^* .

An important difference from the classical GA described in Section 2.3.1 deserves attention: Algorithm 1 does not iterate the selection, mutation, and replacement cycle over multiple generations until the population converges. Instead, it executes a single generation, evaluating the initial population P_0 with a reduced number of epochs E_{fit} only

Algorithm 1 Neuroevolution of the PINN architecture

Require: Population size M ; number of selected individuals K ; screening epochs E_{fit} ; re-evaluation epochs E_{term} ; bounds on the number of hidden layers and of neurons per layer

Ensure: Selected architecture A^*

- 1: Randomly generate the initial population $P_0 = \{A_1, \dots, A_M\}$ within the bounds on layers and neurons
 - 2: **for** each individual $A_i \in P_0$ **do**
 - 3: Train the PINN with architecture A_i for E_{fit} epochs
 - 4: Compute $f(A_i)$ (Equation 2.12)
 - 5: **end for**
 - 6: Sort P_0 by increasing *fitness* and obtain P_0^{select} , the K best individuals (Equation 2.13)
 - 7: **for** each individual $A_i \in P_0^{select}$ **do**
 - 8: Generate the offspring $A'_i = \mu(A_i)$ by mutation (Equation 2.14)
 - 9: **end for**
 - 10: Form the new population $P_1 = P_0^{select} \cup \{A'_1, \dots, A'_K\}$ (Equation 2.15)
 - 11: **for** each individual $A_i \in P_1$ **do**
 - 12: Train the PINN with architecture A_i for E_{term} epochs
 - 13: Compute $f(A_i)$ (Equation 2.12)
 - 14: **end for**
 - 15: $A^* \leftarrow \arg \min_{A_i \in P_1} f(A_i)$
 - 16: **return** A^*
-

for screening, and re-evaluating the resulting population P_1 with a larger number of epochs E_{term} to obtain a more reliable *fitness* before the final choice. This design choice stems from the computational cost of training a PINN: unlike problems in which the *fitness* is cheap to compute and allows thousands of generations, each evaluation here requires training a neural network for a non-trivial number of epochs, which makes a GA with many generations computationally infeasible for this work. This issue is revisited in more detail in Section 2.3.9.

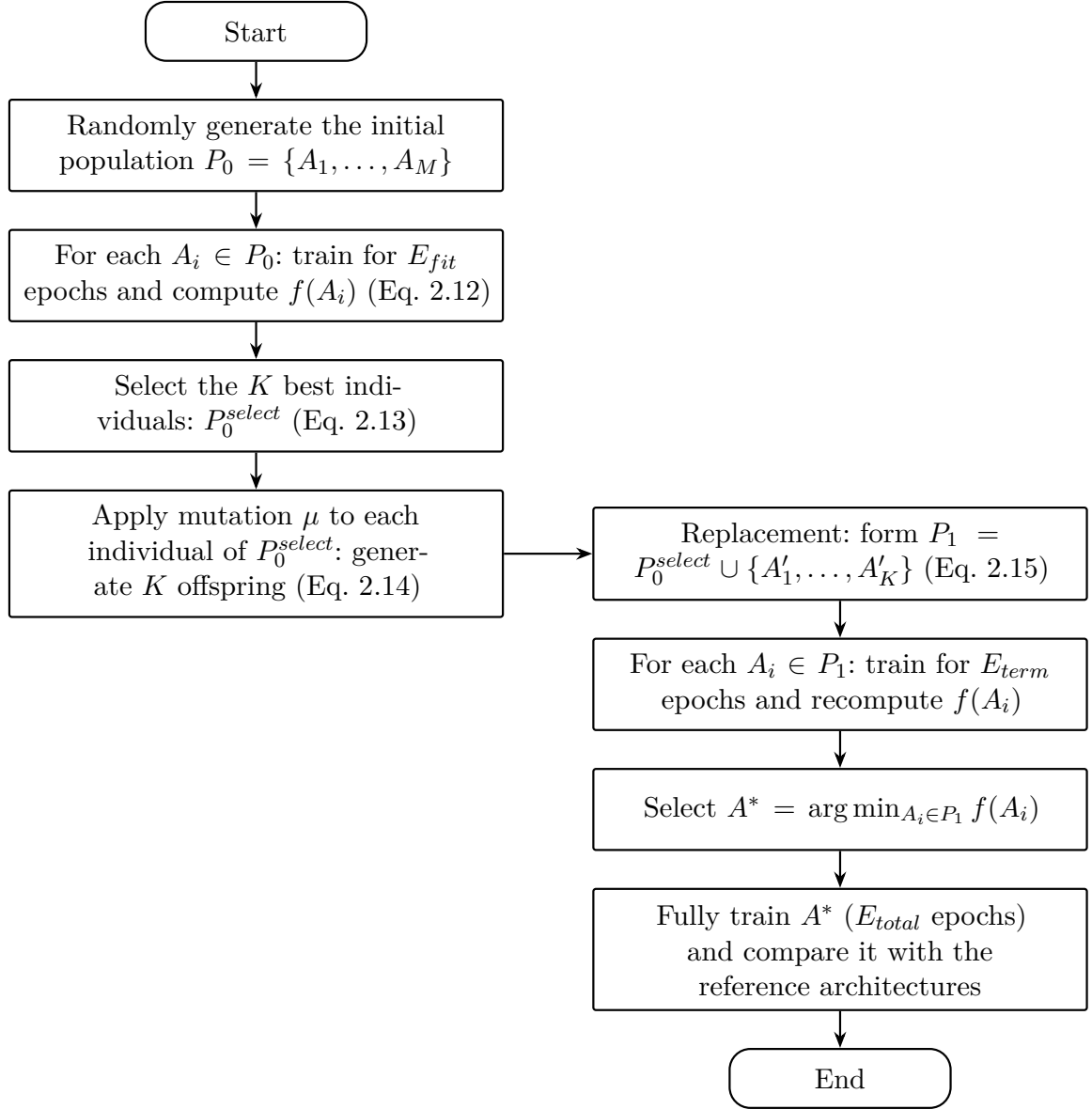
At the end of Algorithm 1, the architecture A^* is fully trained and compared with two reference architectures, as described in Sections 2.3.10 and 4.3.

2.3.7 Neuroevolution Methods in the Literature

As discussed in Section 2.3.2, the neuroevolution methods proposed in the literature can be broadly grouped according to the aspect of the neural network they evolve: weights, topology, or both simultaneously.

The first works on neuroevolution focused on directly evolving the weights of fixed architectures, treating network training as an optimization problem to be solved by an evolutionary algorithm instead of gradient descent. The survey by Yao (1999) extensively catalogs this line of research and observes that, although competitive for small networks or for problems in which the gradient is not available, this approach scales poorly with

Figure 1 – Flowchart of the neuroevolution of the PINN architecture.



Source: Prepared by the author.

the number of parameters and loses competitiveness against gradient descent in larger networks.

Among the methods that evolve the topology of the network, the most influential is NEAT, proposed by Stanley and Miikkulainen (2002). The motivation of NEAT is a critique of manual architecture design: instead of fixing the topology of the network a priori and evolving only its weights, or of searching in a manually defined space of architectures, NEAT evolves weights and topology simultaneously, starting from minimal networks without hidden layers and gradually complexifying them through structural mutations that add nodes and connections. This progressive complexification avoids exploring an unnecessarily large space of architectures from the start. To make this process possible,

NEAT introduces two central mechanisms. The first is the historical marking of genes through innovation numbers, which identifies genes of common origin among individuals with different topologies and makes a consistent crossover between them possible, solving the problem of competing conventions, i.e., the difficulty of consistently recombining networks with different topologies. The second is speciation, which groups topologically similar individuals into subpopulations and shares the *fitness* among the members of each species, protecting recent structural innovations from immediate competition with the rest of the population until their weights have had a chance to be optimized.

Not all topology-evolving methods rely on the incremental complexification and speciation of NEAT. A simpler alternative, adopted by Akut and Kulkarni (2019) for the automatic design of deep learning architectures, is to directly encode high-level structural characteristics, such as the number of layers and of neurons per layer, in a short chromosome and to search with a conventional GA for the combination that yields the best performance, without progressive complexification, speciation, or innovation history.

The method used in this work, detailed in Section 2.3.10, is closer to this second category: the number of hidden layers and of neurons per layer is evolved directly, without the speciation and historical marking mechanisms of NEAT, and without crossover, using only mutation, as discussed in Section 2.3.5. Works that apply neuroevolution specifically to the optimization of PINNs, including the architecture search problem addressed in this monograph, are discussed separately in Chapter 3.

2.3.8 Neuroevolution and Gradient-Based Training

Training a neural network by gradient descent, described in Section 2.1, requires the loss function to be differentiable with respect to the parameters to be optimized, since the backpropagation algorithm computes precisely the gradient of this function with respect to these parameters. This requirement is met by the weights and biases of a fixed architecture, but not by structural choices such as the number of hidden layers or the number of neurons per layer: there is no gradient of the loss function with respect to a discrete quantity such as the “number of layers”, since changing this value changes the very structure of the computational graph of the network, not just the value of a continuous parameter within it. This limitation is what makes gradient descent inapplicable to architecture search and motivates the use of a method such as neuroevolution, which does not require differentiability with respect to the evolved parameters (Yao, 1999).

Besides naturally handling discrete parameters, evolutionary search keeps and evaluates, at each generation, a whole population of candidate solutions, instead of following a single trajectory guided by the gradient from a starting point. This allows different regions of the search space to be explored simultaneously, which can be advantageous in spaces with multiple local optima or in problems in which the relation between the

structural choice and the final quality of the network is not smooth. In the specific context of PINNs, Sung *et al.* (2023) show, by comparing neuroevolution algorithms with stochastic gradient descent, that evolutionary methods can even outperform gradient descent in how well the learned solution complies with the differential equation that governs the physical problem, as discussed in more detail in Chapter 3.

These advantages, however, come at a cost. Each *fitness* evaluation in a neuroevolution algorithm requires training, even if only partially, the network corresponding to each individual of the population, which is orders of magnitude more expensive than a single weight update by backpropagation. While gradient descent uses the direction information provided by the gradient to converge directly to a local minimum of the loss function, an evolutionary search applied to the weight space must explore it essentially blindly, without direction information, which makes it uncompetitive for directly optimizing the weights of networks with many parameters, as discussed in Section 2.3.7.

It is precisely this complementarity, and not a relation of substitution, that guides the methodological choice of this work: gradient descent, through the Adam optimizer (Kingma; Ba, 2014), remains responsible for training the weights of each candidate architecture, taking advantage of its efficiency in continuous and differentiable parameter spaces, while neuroevolution is reserved exclusively for the search for the architecture, a discrete and non-differentiable space in which gradient descent cannot be applied directly.

2.3.9 Complexity and Limitations

Unlike training by gradient descent, whose cost per iteration is approximately constant and known in advance, the computational cost of a neuroevolution algorithm is dominated by the number of *fitness* evaluations, each of which requires training, even if partially, a different neural network. In Algorithm 1, the initial population P_0 , with M individuals, is evaluated once with E_{fit} epochs, and the population resulting from the replacement, P_1 , also with M individuals, is evaluated again with E_{term} epochs. The total cost of the evolutionary search therefore corresponds to $2M$ partial PINN trainings, considerably fewer than those required by an exhaustive search, which would fully train each candidate architecture considered along the process.

This cost is directly proportional to the population size M and to the number of selected individuals K : increasing M broadens the diversity of architectures explored in the initialization, at the cost of more *fitness* evaluations; increasing K preserves more individuals of the original population and generates more offspring per generation, at the cost of reducing the selective pressure, since a larger fraction of the population survives regardless of its relative performance. The values of M and K adopted in this work are given in Section 5.2.

The classical notion of convergence of a GA, associated with the stabilization of

the *fitness* of the population over many generations (Eiben; Smith, 2015), does not directly apply to the method used in this work, since, as discussed in Section 2.3.6, only a single generation of selection and mutation is executed, for computational cost reasons. This means that the method does not seek to make the population converge to a specific region of the space of architectures over time; instead, it performs a single refinement of the initial population, whose quality strongly depends on that initial population already containing, by random draw, reasonably promising architectures.

This limitation is directly related to the diversity of the population. Since the mutation operator defined by Equation 2.14 applies only non-negative increments, the search tends to explore only architectures equal to or larger than those of the initial population, never smaller, introducing a directional bias in the exploration of the search space. Moreover, since the offspring are generated exclusively from the K best individuals of P_0 , and the method does not include diversity-preservation mechanisms such as the speciation used by NEAT (Section 2.3.7), the exploration is restricted to a relatively narrow neighborhood of these individuals. Unlike the classical risk of premature convergence, in which diversity is lost over many generations, the risk here is more immediate: with a single generation and no reintroduction of diversity, an initial population that poorly represents the space of architectures can compromise the quality of the final architecture A^* , without the algorithm having the chance to recover from this choice over subsequent generations.

2.3.10 Neuroevolution Applied to This Work

This section gathers, qualitatively, the design choices that instantiate Algorithm 1 for the specific problem of this work: the search for the architecture of a PINN. The concrete values corresponding to each choice are given in Section 5.2, together with the other parameters of the experiment.

Each individual represents a candidate PINN architecture, encoded by the chromosome defined in Equation 2.11: the number of hidden layers L_i and the number of neurons n_i , equal in all hidden layers. The weights and biases of the network are not part of the chromosome; they are obtained, for each candidate architecture, by training with the Adam optimizer (Kingma; Ba, 2014), as discussed in Section 2.3.8. In other words, the only parameter effectively evolved by the genetic algorithm is the architecture of the network; the weights are treated as a consequence of the training of each architecture, not as genetic material.

The *fitness* of each individual is computed according to Equation 2.12: the candidate architecture is trained for a reduced number of epochs, and its *fitness* is the mean of the total loss over the last epochs of this partial training. Selection follows the truncation strategy defined in Equation 2.13, keeping only the individuals with the best *fitness* for the next step.

The mutation operator, defined in Equation 2.14, is applied with probability 1 to each selected individual, generating one offspring per individual through random non-negative increments in the number of layers and in the number of neurons per layer. There is therefore no per-gene mutation rate in the classical sense (Section 2.3.5): every selected individual is mutated, and the variation lies in the random magnitude, not in the probability, of the applied increment.

Replacement follows the elitist strategy defined in Equation 2.15: the offspring take the place of the individuals with the worst *fitness* in the original population, while the selected individuals are preserved unchanged, forming the new population. The stopping criterion, as discussed in Section 2.3.6, is not iterative: this new population is fully re-evaluated with a larger number of epochs than in the initial *fitness* step, and the best network at the end of the evolution is obtained by selecting, among the re-evaluated individuals, the architecture A^* with the lowest *fitness*.

Finally, A^* is fully trained and compared with two reference architectures, also fully trained: a randomly drawn architecture and the best architecture of a random search with the same epoch budget, following the validation strategy presented in Section 4.3. The concrete configuration of this procedure, including the population size, the bounds on layers and neurons, the number of epochs of each step, and the number of experimental runs, is presented in Section 5.2, together with the results obtained for the three physical problems studied.

3 RELATED WORK

3.1 Physics Informed Neural Networks

Since their proposal by Raissi, Perdikaris and Karniadakis (2019), PINNs have been applied to a wide range of problems in fluid mechanics, heat transfer, electromagnetism, and quantum mechanics, among other areas of physics and engineering. The survey by Karniadakis *et al.* (2021), published in *Nature Reviews Physics*, consolidates the main advances of physics-informed machine learning and discusses the incorporation of physical laws both in the loss function, the approach adopted in this work, and in other components of the model. The review by Cuomo *et al.* (2022) extensively catalogs the PINN variants proposed in the literature and shows that, despite the conceptual simplicity of PINNs, their effective implementation involves several non-trivial design choices, among them the architecture of the neural network, which is the central focus of this work.

3.2 Neuroevolution in Neural Networks and PINNs

Neuroevolution is an established research area for traditional neural networks, used both to optimize weights and to evolve network topologies (Stanley *et al.*, 2019). Akut and Kulkarni (2019) demonstrate the use of genetic algorithms for the automatic design of *deep learning* architectures and report performance gains over manually designed architectures.

In the specific context of PINNs, the combination with evolutionary algorithms is still recent but already shows promising results. Kaplarević-Mališić *et al.* (2023) propose PINN/GA, a genetic algorithm *framework* with selection, crossover, and mutation operators specially adapted to the search for architectures and hyperparameters of PINNs. The authors validate the approach on two problems: the one-dimensional Stefan problem and the Helmholtz equation. They report that the average error of the PINN decreases by about one order of magnitude along the evolution, that the proposed distributed design speeds up the hyperparameter search by almost two orders of magnitude, and that PINN/GA outperforms a parallel random search and the *Tree of Parzen Estimators* method of Hyperopt run in the same computational environment.

Sung *et al.* (2023) propose a set of five benchmark problems: the convection–diffusion equation, projectile motion, the Korteweg–de Vries equation, and the linear and nonlinear Burgers’ equations, to compare neuroevolution algorithms (CMA-ES and xNES+NAG) with stochastic gradient descent (SGD) for training PINNs. The authors assess neuroevolution by comparing the test MSE and the degree of compliance of the solution with the differential equation (physics residual) across methods, and show that evolutionary algorithms can outperform gradient descent in ensuring that the learned

solution obeys the physical laws.

Outside the context of PINNs, the neural architecture search (*Neural Architecture Search*) literature shows that random search is a strong baseline that is often hard to beat. Li and Talwalkar (2020) show that random search with the same budget is competitive with much more sophisticated architecture search methods and argue that every new search strategy should be compared with it, which motivated the inclusion of this reference in the evaluation of this work.

Finally, the *survey* by Wong *et al.* (2026) consolidates these and other related works and points to the combination of neuroevolution and PINNs as one of the most promising frontiers of scientific machine learning. These works, however, either use neuroevolution to train the weights of the PINN (Sung *et al.*, 2023) or run a complete evolutionary search, with many generations and a large distributed computational budget (Kaplarević-Mališić *et al.*, 2023). This work investigates a minimal, low-cost variant, with a single generation guided by partial trainings, and measures, over many independent and reproducible runs, whether it selects architectures better than both a random choice and a random search with the same budget, which characterizes the contribution of this monograph.

4 METHODOLOGY

4.1 Overview of the Method

This work proposes applying the concept of neuroevolution to the definition of the architecture of PINNs, in two distinct stages: (i) an evolutionary search stage, in which a genetic algorithm explores the space of possible architectures, i.e., the number of hidden layers and of neurons per layer, evaluating each candidate through a short, partial training as a PINN; and (ii) a full training stage, in which the architecture selected at the end of the evolutionary process is fully trained as a PINN and then compared with two reference architectures, also fully trained: a randomly chosen architecture and the best architecture found by a random search with the same epoch budget as the evolutionary search. This design makes it possible to assess not only whether the evolved PINN adequately solves the physical problem, but also whether the evolutionary search is indeed able to identify architectures better than a random choice, without exhaustively training all candidate architectures, and whether this gain is due to the evolutionary operators or merely to evaluating more than one candidate.

4.2 Genetic Algorithm

The individual representation, the *fitness* function, the evolutionary operators, and the neuroevolution algorithm used in this work to search for the architecture of the PINN were formalized in Sections 2.3.3 to 2.3.6 of Chapter 2 (Equations 2.11 to 2.15, and Algorithm 1). This procedure is applied identically to the three physical problems studied in this work (pendulum, heat transfer in a one-dimensional bar, and vibrating string): in all cases, the population consists of candidate PINN architectures, encoded according to Equation 2.11, and the search follows Algorithm 1 exactly.

At the end of the evolutionary search, the architecture A^* selected by Algorithm 1 is trained for the total of E_{total} epochs of a PINN training ($E_{total} > E_{term} > E_{fit}$) and compared with the reference architectures described below, also trained for the same total number of epochs. The concrete values adopted for the population size, the bounds on layers and neurons, the number of epochs of each stage, and the number of experimental runs, identical for the three problems, are presented in Section 5.2.

4.3 Validation Strategy

4.3.1 Comparison Arms

Each run of the experiment compares three arms, all ending with the full training, for E_{total} epochs, of the chosen architecture:

1. **GA**: the architecture A^* selected by Algorithm 1, whose search consumes $M \cdot E_{fit} + M \cdot E_{term}$ training epochs;
2. **Random**: a single architecture drawn within the same bounds on layers and neurons as the initial population of the GA, without any search;
3. **Random search**: M architectures drawn within the same bounds, each trained for $(E_{fit} + E_{term})$ epochs and ranked by the same *fitness* (Equation 2.12); the best one is chosen. This search consumes exactly the same epoch budget as the GA.

The random arm measures the gain of performing any search over performing none. Random search with the same budget, in turn, isolates the contribution of the evolutionary operators (selection, mutation, and re-evaluation) from the effect of simply evaluating more than one candidate, following the recommendation of the neural architecture search literature to compare every new search strategy with this baseline (Li; Talwalkar, 2020).

4.3.2 Metrics

Since the three problems studied have known exact solutions (Chapter 5), the primary metric is the relative L^2 error of the trained PINN with respect to this solution,

$$\varepsilon_{L^2} = \frac{\|\hat{u} - u\|_2}{\|u\|_2} = \sqrt{\frac{\sum_j (\hat{u}(x_j) - u(x_j))^2}{\sum_j u(x_j)^2}}, \quad (4.1)$$

computed on an evaluation grid independent of the points used in training. This metric directly measures what is expected from a PINN, namely to approximate the solution of the physical problem, and does not depend on the scale of the terms of the loss function. The secondary metric is the final loss, defined as the mean of the total loss over the last 100 epochs of the full training, which smooths the oscillations of the optimizer. For both metrics, lower values indicate a better result.

4.3.3 Statistical Analysis

The complete experiment is repeated in several independent runs. In each run, the three arms are executed with seeds derived from the same master seed, so the comparison between them is **paired**: the GA of a run is compared with the references of the same run. For each comparison between the GA and a reference, we compute:

- the number of runs in which the GA reached the lower error (wins), with the Wilson 95% confidence interval for the win rate (Wilson, 1927);
- a one-sided exact binomial test, whose null hypothesis is that the GA and the reference are equally likely to win (0.5);

- a one-sided Wilcoxon signed-rank test (Wilcoxon, 1945), applied to the paired differences $d = \ln(\varepsilon_{\text{ref}}/\varepsilon_{\text{GA}})$, which considers not only the sign but also the size of each difference and does not require the data to follow a normal distribution;
- the median ratio $\varepsilon_{\text{ref}}/\varepsilon_{\text{GA}}$, with a 95% confidence interval obtained by bootstrap (Efron, 1979); values above 1 favor the GA.

The logarithm is used because the errors of PINNs span several orders of magnitude: with it, an error two times smaller and an error two times larger have the same weight, with opposite signs. Since six comparisons are made for each test (three problems and two references), the p -values are adjusted by the Holm procedure (Holm, 1979), which keeps at 5% the probability of at least one false positive across the comparisons. A significance level of 5% is adopted for the adjusted p -values.

Finally, to verify the hypothesis on which the method relies, namely that the loss after a partial training ranks the candidates similarly to the error, we compute the Spearman rank correlation (Spearman, 1904) between the *fitness* and the relative L^2 error of the candidates evaluated in the screening of the GA.

4.4 Reproducibility

All random decisions of a run r (initial population, mutations, reference architectures, and weight initialization of each network) are derived from a master seed $1000 + r$ through the `SeedSequence` generator of the NumPy library, and the derived seeds are recorded together with the results of the run. The trainings use deterministic PyTorch algorithms, so each run can be reproduced individually on the same *hardware* and with the same *software* versions, which are also recorded. The configuration of each experiment is kept in text files, separate from the code, and each run stores the complete configuration, the loss history of all trainings, and the final weights of the networks. The code, the results, and a set of automated tests, which verify, among other things, that the exact solutions satisfy the equations, that the derivatives computed by automatic differentiation match the analytical ones, and that the networks used have non-zero second derivatives, are publicly available¹.

¹ <https://github.com/GMCavalheri/MBA-PINN-Neuroevolution>

5 EXPERIMENTAL EVALUATION

5.1 Computational Setup

The method was implemented in Python 3.12, with the libraries PyTorch 2.11 (with CUDA 12.8), NumPy, SciPy, pandas, and Matplotlib. PyTorch was used to build and train the neural networks and the PINNs and to compute, by automatic differentiation (`torch.autograd.grad`), the derivatives of the network with respect to its inputs that make up the residual of the differential equations. SciPy was used for the statistical tests, pandas for organizing the results, and Matplotlib for generating all the figures of this work.

The experiments were run on a laptop with an Intel Core i7-10750H processor (2.60 GHz, 6 cores) and an NVIDIA GeForce RTX 2060 Mobile graphics card (6 GB). Because the networks evaluated are small, the training time on the GPU is dominated by the launch of many small operations rather than by the computation itself. For this reason, each training step, which includes computing the loss, the derivatives of the network with respect to its inputs, the backpropagation, and the Adam update, is captured once in a CUDA graph and then replayed, which reduced the time per epoch by a factor of 3 to 13 in a preliminary test. Automatic mixed precision training (with 16-bit floating-point numbers) was also evaluated, but it was slower and increased the relative L^2 error by more than 10% on the PDE problems, which depend on accurate second derivatives; therefore, all trainings use single precision (32 bits). Three runs were executed in parallel on the GPU.

5.2 Neuroevolution Configuration

The neuroevolution configuration, described qualitatively in Section 2.3.10, was applied identically to the three physical problems studied in this work (pendulum, heat transfer, and vibrating string) and is summarized in Table 1. We adopted a population of $M = 10$ architectures, randomly generated in the population initialization step within the following bounds: between 1 and 10 hidden layers and between 1 and 20 neurons per layer. All networks use the hyperbolic tangent as activation function, Xavier initialization (Géron, 2019), and the Adam optimizer (Kingma; Ba, 2014).

In the *fitness* step, each of the $M = 10$ architectures was trained for $E_{fit} = 1,000$ epochs, equivalent to 5% of the total of $E_{total} = 20,000$ training epochs, and its *fitness* $f(A_i)$ was computed as the mean of the total loss over the last $W = 100$ epochs (Equation 2.12). In the selection step, the $K = 5$ architectures with the lowest *fitness* were selected.

In the mutation step, the operator μ was applied to the $K = 5$ selected architectures, generating $K = 5$ new offspring, with increments Δn_i of 0, 1, or 2 neurons, applied equally

to all hidden layers, and ΔL_i of 0 or 1 additional layer. The bounds of 1 to 10 layers and 1 to 20 neurons apply only to the initial population; an offspring can therefore exceed them by one layer or by up to two neurons. In the replacement step, the offspring replaced the $K = 5$ worst architectures of the original population, keeping the population size at $M = 10$ individuals.

Finally, the stopping criterion established that the 10 architectures resulting from the replacement step were trained for $E_{term} = 2,000$ epochs (10% of the total number of training epochs), and the architecture with the lowest *fitness* at the end of this step was selected as the final solution A^* . The complete search therefore consumes $10 \times 1,000 + 10 \times 2,000 = 30,000$ epochs, equivalent to about 1.5 full trainings.

The architecture A^* was then trained for the total of $E_{total} = 20,000$ epochs and, according to the validation strategy presented in Section 4.3, compared with a randomly drawn architecture and with the best of 10 randomly drawn architectures trained for 3,000 epochs each (random search), which consumes the same budget of 30,000 epochs; both were also trained for E_{total} epochs. This complete experiment was repeated in 35 independent runs, with master seeds from 1000 to 1034, for each of the three physical problems studied.

Table 1 – Configuration of the experiments, identical for the three problems.

Parameter	Value
Population size (M)	10
Selected individuals (K)	5
Hidden layers (initial population)	1 to 10
Neurons per layer (initial population)	1 to 20
Mutation increment, layers (ΔL)	0 or 1
Mutation increment, neurons per layer (Δn)	0, 1, or 2
Screening epochs (E_{fit})	1,000
Re-evaluation epochs (E_{term})	2,000
Full training epochs (E_{total})	20,000
<i>Fitness</i> window (W)	last 100 epochs
Random search (reference)	10 architectures $\times$ 3,000 epochs
Activation function	hyperbolic tangent
Independent runs per problem	35 (seeds 1000 to 1034)

Source: Prepared by the author.

5.3 Pendulum

In this work, the motion of the simple one-dimensional pendulum, without damping or external force, in the small-angle approximation, is described by the ODE below

$$\frac{d^2x}{dt^2} = \ddot{x} = -\omega_0^2 \times x; \quad (5.1)$$

$$g(x, t) = \ddot{x} + \omega_0^2 x = 0, \quad (5.2)$$

where ω_0 is the angular frequency and $\ddot{x}$ is the second derivative of x with respect to t . This equation has been widely studied in the literature and has the following analytical solution (Thornton; Marion, 2004):

$$x(t) = A \times \sin(\omega_0 t - \delta), \quad (5.3)$$

where A is the amplitude of oscillation and δ is the phase constant. For the one-dimensional pendulum, the number of output neurons and target attributes is 1, the position of the pendulum along the x axis, $x(t)$. The number of input neurons and attributes is 1, the time t , which is the independent variable.

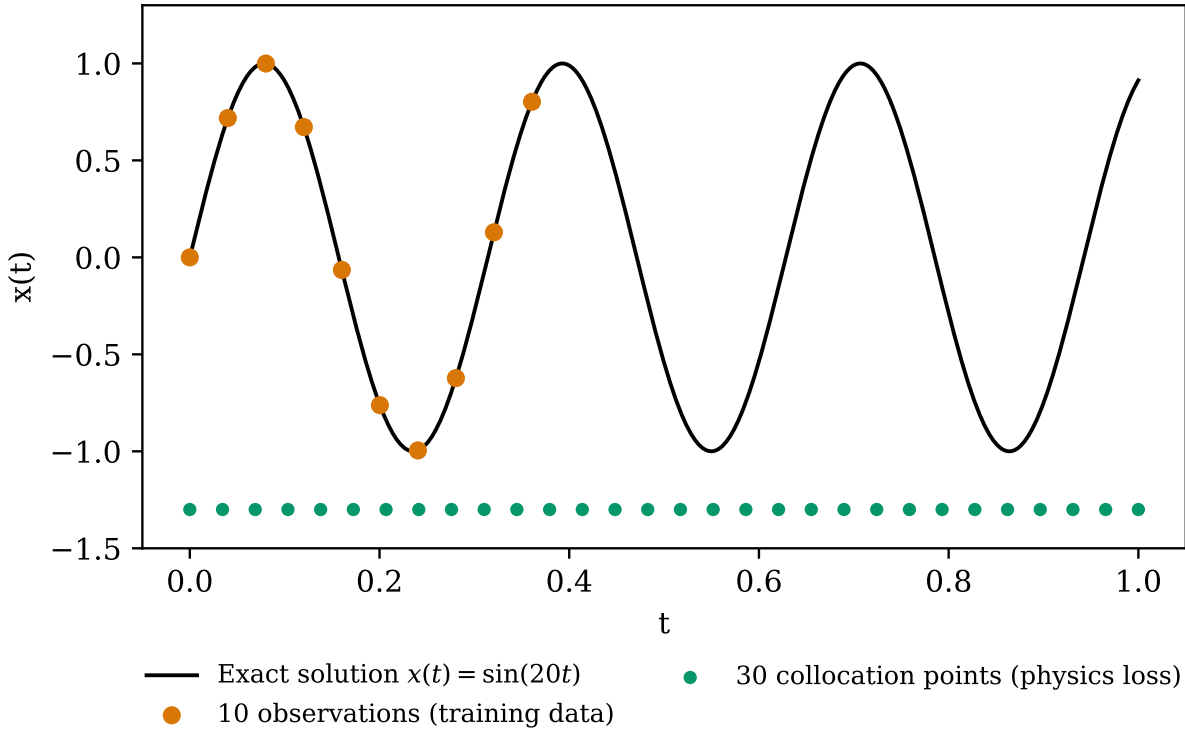
We used the parameters $A = 1$, $\delta = 0$, $\omega_0 = 20$, and $t \in [0, 1]$, so that the exact solution is $x(t) = \sin(20t)$, with about 3.2 oscillations in the interval. The time domain was divided into 500 equally spaced points, of which only 10, equally spaced at the beginning of the domain ($t < 0.4$), are used as training data for the data loss. The physics loss is computed at 30 equally spaced collocation points over the whole interval $[0, 1]$, as shown in Figure 2. The loss of the PINN is $\mathcal{L}_{\text{data}} + \lambda \mathcal{L}_{\text{phys}}$, with $\lambda = 10^{-4}$: since the residual of Equation 5.2 has a scale set by $\omega_0^2 = 400$, this weight balances the contributions of the two terms. The relative L^2 error is computed at 1,000 equally spaced points in $[0, 1]$.

5.3.1 Standard Neural Network and PINN

To illustrate the need for a PINN architecture instead of a standard one, two neural networks with the same structural architecture were trained to solve the pendulum problem: one uses the MSE of the data and the physical contribution of the problem, which characterizes it as a PINN, and the other uses only the MSE of the data, which characterizes it as standard. Both networks have 3 hidden layers with 32 fully connected neurons each, one neuron in the input layer and one in the output layer, hyperbolic tangent activations, and the same weight initialization (fixed seed). The input of the network is the time t , the independent variable, and the output is the position x , the dependent variable. The optimization algorithm is Adam (Kingma; Ba, 2014).

The standard neural network was trained with the MSE loss only, for 1,000 epochs, with a learning rate of 10^{-3} , with the results shown in Figure 3. After 200 epochs, the network has not yet fitted the data; after 1,000 epochs, it interpolates the 10 training points but, outside them, follows a trend unrelated to the physical phenomenon, with a relative L^2 error of 5.6 over the whole interval.

Figure 2 – Exact solution of the pendulum, training data, and collocation points of the physics loss.



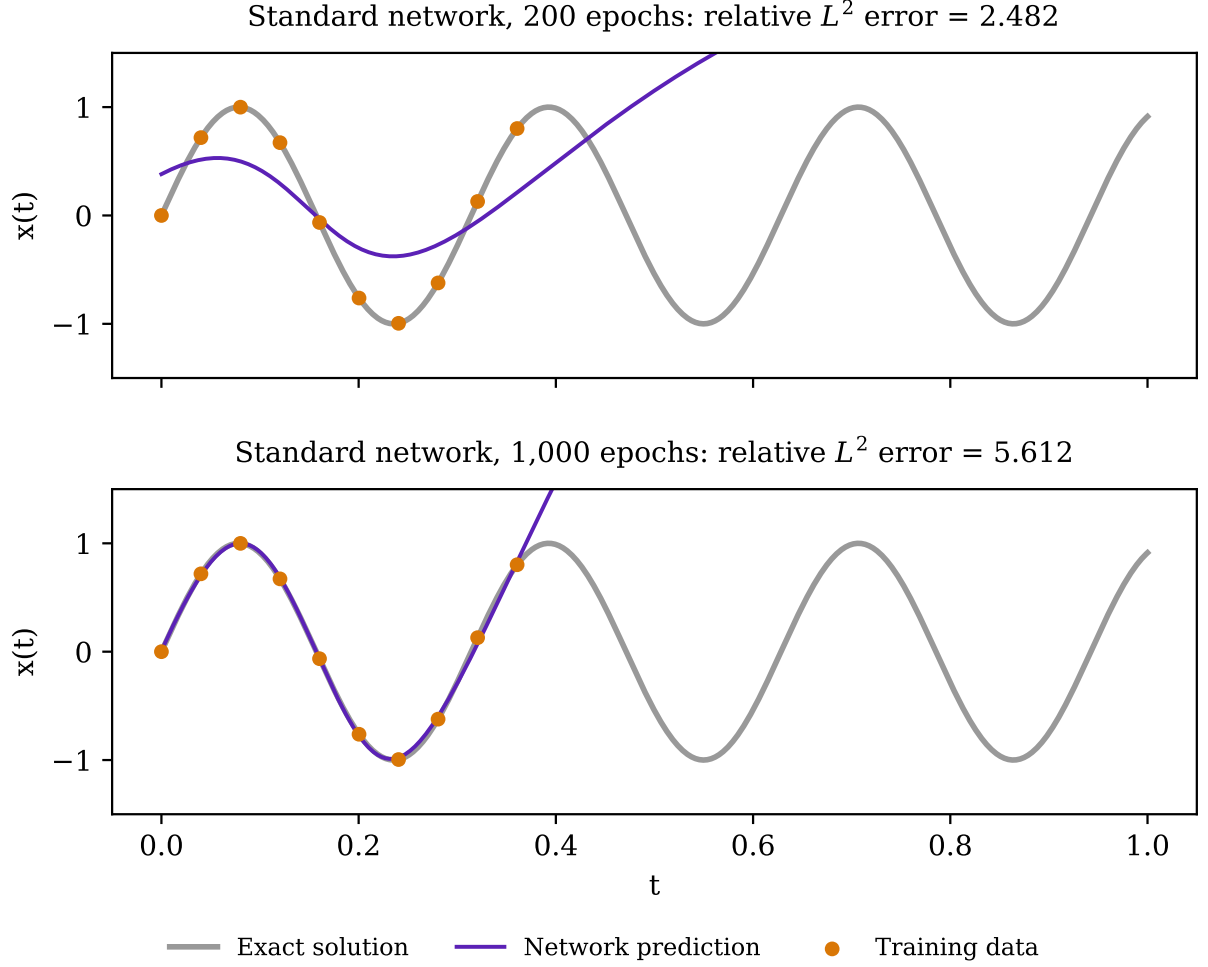
Source: Prepared by the author.

Next, the PINN was trained with the same architecture and the same training data, but with the physics loss added, a learning rate of 10^{-4} , and 20,000 epochs. The PyTorch library provides the `autograd.grad` function, which computes, by automatic differentiation, the derivatives of the output of the network with respect to its inputs; applied twice, it yields $\ddot{x}$, used in the residual of Equation 5.2. Figure 4 shows the predictions of the PINN at epochs 6,000, 12,000, 18,000, and 20,000.

The figures show that the PINN needs more epochs to interpolate the training data than the standard neural network. However, the gain in generalization due to the physics loss is much more important and happens alongside the interpolation of the data: at the end of training, the PINN reproduces the oscillation over the whole interval, including the region without data, with a relative L^2 error of 0.003, about 2,000 times smaller than that of the standard network.

The failure of the standard model to generalize stems from its loss function referring only to the data of the problem, and not to the kind of physical phenomenon being analyzed. An MSE loss function used alone cannot identify which physical problem is being addressed: if the data came from any other area of physics, it would simply try to minimize them, without capturing the underlying dynamics of the system.

Figure 3 – Values predicted by the standard neural network after 200 and 1,000 training epochs.



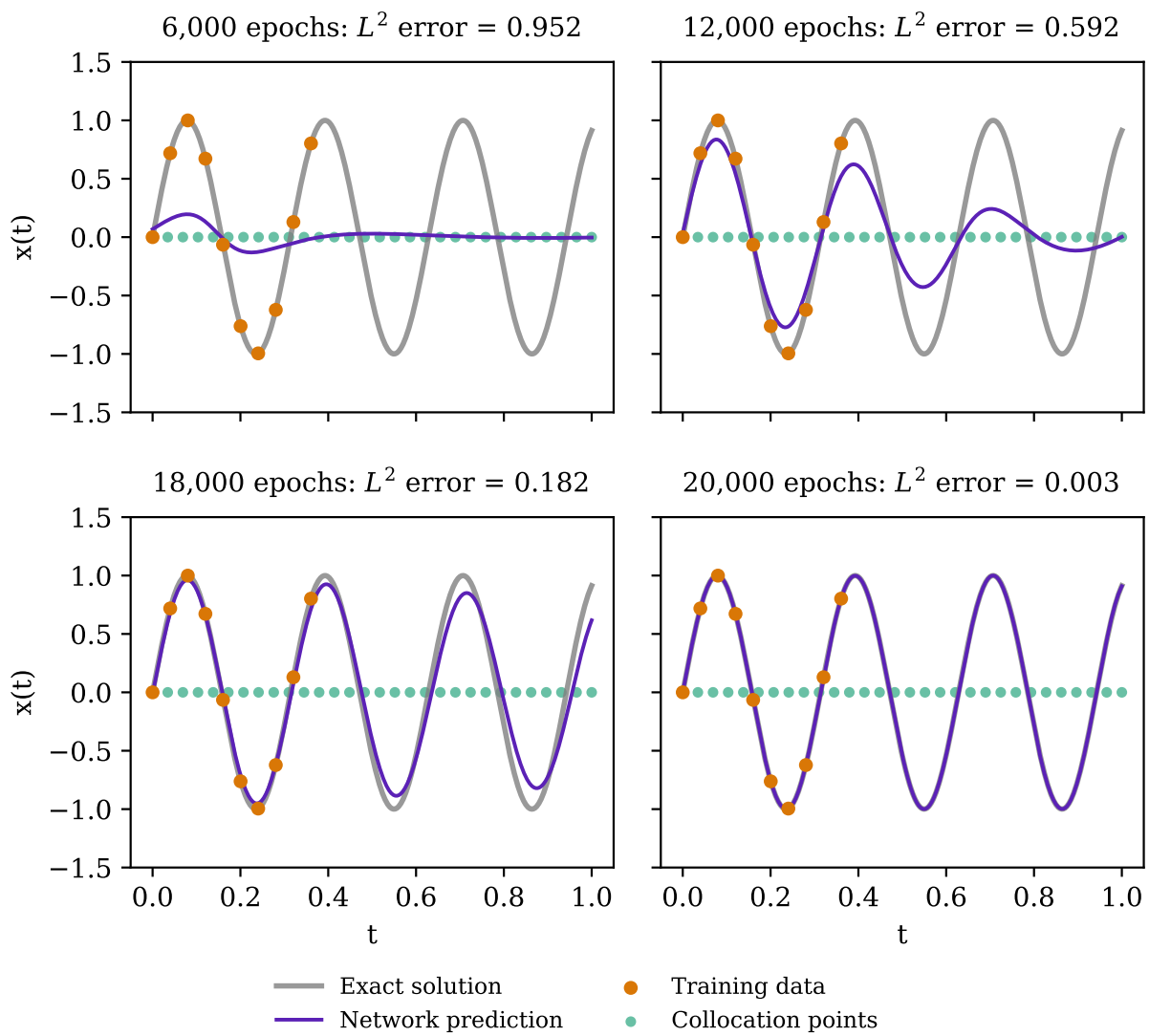
Source: Prepared by the author.

Figure 5 shows the evolution of the loss functions of the two networks. In the standard network, the loss barely decreases in the first 200 epochs, then drops very sharply, followed by a slow decrease until epoch 1,000, which shows that the model quickly adapts to the training data. In the PINN, the two terms of the loss compete with each other. The physics term drops quickly in the first epochs, because a network with output close to zero trivially satisfies Equation 5.2; as the network starts fitting the data and oscillating, the physics term grows again, until, from about 17,000 epochs on, the network finds an oscillation that satisfies both the data and the equation, and both terms drop by several orders of magnitude.

5.3.2 Example of a Neuroevolution Run

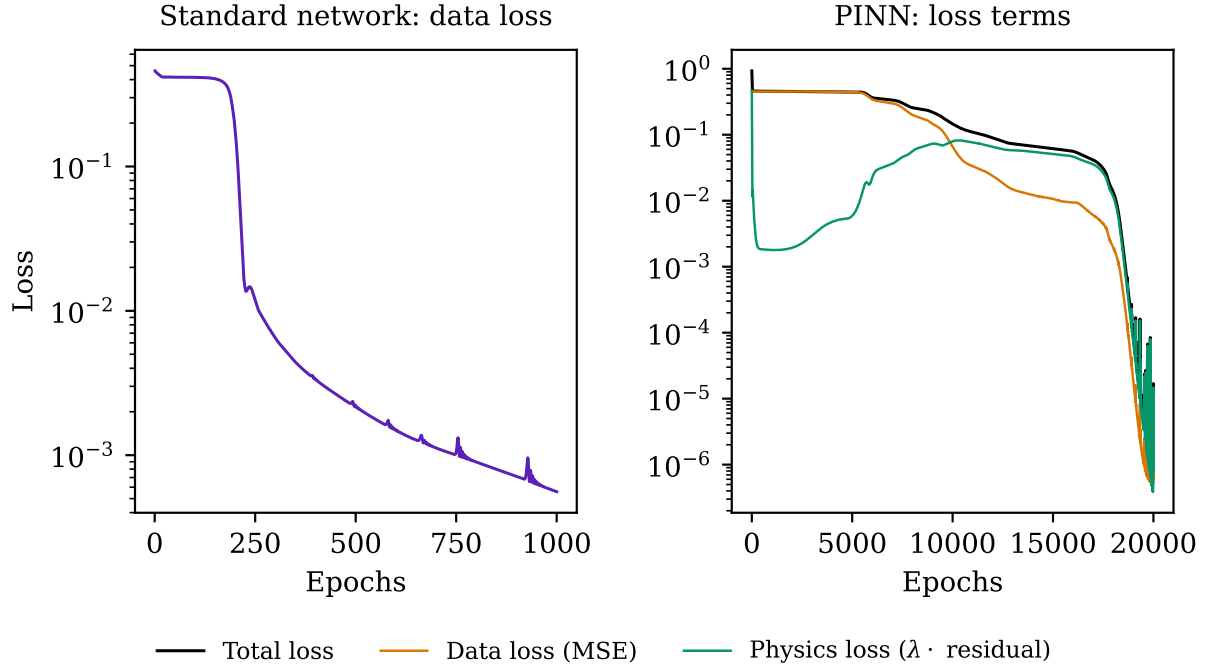
To illustrate how Algorithm 1 works, Table 2 presents run 5 of the pendulum, whose PINN obtained with the GA architecture had the median error among the 35 runs.

Figure 4 – Values predicted by the PINN after 6,000, 12,000, 18,000, and 20,000 training epochs.



Source: Prepared by the author.

Figure 5 – Loss of the standard neural network and terms of the loss of the PINN, as a function of the training epochs.



Source: Prepared by the author.

Of the 10 architectures of the initial population, trained for 1,000 epochs, the 5 with the lowest *fitness* were selected, and each generated one offspring by mutation.

Table 2 – The 5 architectures selected in the screening of run 5 of the pendulum, with their *fitness* after E_{fit} epochs, the offspring generated by mutation, and the *fitness* of both after E_{term} epochs. Architectures are given as (hidden layers, neurons per layer).

Selected	<i>Fitness</i> (E_{fit})	<i>Fitness</i> (E_{term})	Offspring	<i>Fitness</i> (E_{term})
(7, 15)	0.4514	0.2420	(8, 16)	0.3636
(10, 3)	0.4521	0.4512	(11, 5)	0.4565
(2, 15)	0.4559	0.4536	(2, 16)	0.4566
(10, 2)	0.4562	0.4213	(11, 4)	0.3304
(6, 12)	0.4566	0.4552	(7, 14)	0.4438

Source: Prepared by the author.

After 1,000 epochs, the *fitness* values of the candidates differ only in the third decimal place: in the pendulum, with a learning rate of 10^{-4} , the networks are still on the initial plateau of training (Figure 5), and the screening distinguishes the candidates with little margin. After the re-evaluation with 2,000 epochs, the differences become larger, and the best architecture was (7, 15) itself, selected in the screening, rather than one of its

offspring. Table 3 compares this architecture with the two references of the same run.

Table 3 – Architectures of the three arms in run 5 of the pendulum and relative L^2 error of the PINN after full training.

Arm	Hidden layers	Neurons per layer	Relative L^2 error
GA	7	15	5.8×10^{-3}
Random	7	7	2.1×10^{-2}
Random search	7	17	3.5×10^{-4}

Source: Prepared by the author.

In this run, the error of the GA architecture was about 3.7 times smaller than that of the randomly drawn architecture, but larger than that of random search, which found an architecture of the same depth and slightly wider. The results of the 35 runs are presented in Section 5.6.

5.4 Heat Transfer in a One-Dimensional Bar

Heat transfer in a one-dimensional bar is described by the PDE below (Hahn; Özisik, 2012)

$$\alpha \frac{\partial^2 u}{\partial x^2} = \frac{\partial u}{\partial t}; \quad (5.4)$$

$$g(x, u, t) = \alpha \frac{\partial^2 u}{\partial x^2} - \frac{\partial u}{\partial t} = 0, \quad (5.5)$$

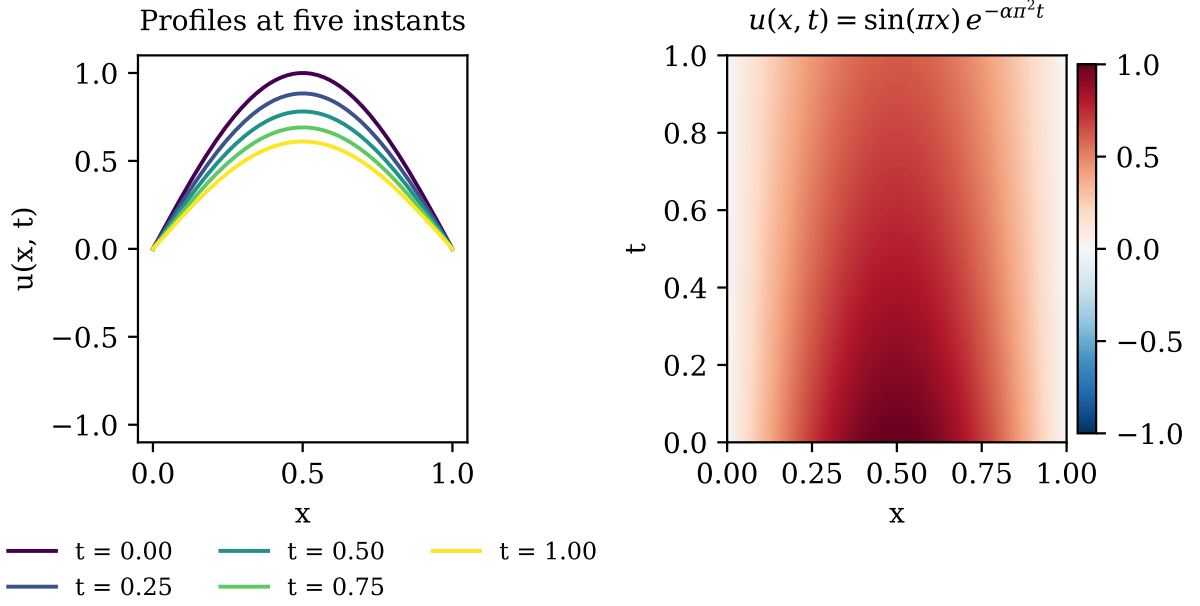
where u is the temperature at position x and time t , and α is the thermal diffusivity, which measures how fast heat propagates through the material. This is one of the most used and studied equations in solid-state physics and thermodynamics. We used the dimensionless problem, with $x, t \in [0, 1]$ and $\alpha = 0.05$, initial condition $u(x, 0) = \sin(\pi x)$, which represents a hot spot at the center of the bar, and zero temperature at the ends, $u(0, t) = u(1, t) = 0$. For these conditions, separation of variables gives the exact solution

$$u(x, t) = \sin(\pi x) e^{-\alpha \pi^2 t}, \quad (5.6)$$

in which the initial profile keeps its shape and decays exponentially in time, as shown in Figure 6.

The input of the network is the position x and the time t , the independent variables, and the output is the temperature u , the dependent variable. The activation function of all layers is the hyperbolic tangent, the optimizer is Adam (Kingma; Ba, 2014), and the learning rate is 10^{-3} . The residual of Equation 5.5 is computed on a uniform grid of

Figure 6 – Exact solution of the heat transfer problem used in this work: temperature profiles at five instants and map of $u(x, t)$.



Source: Prepared by the author.

collocation points with 100 points in x and 250 points in t , for a total of 25,000 pairs (x, t) . The loss function is the sum of four mean squared error terms with equal weights: the residual at the collocation points, the initial condition (100 points at $t = 0$), and the two boundary conditions (250 points at each end). There are no observed data besides the initial and boundary conditions. The relative L^2 error is computed on an independent grid of 201×201 points.

5.5 Vibrating String

The motion of a vibrating string, with transverse displacement u along one spatial dimension x , is described by the wave equation (Thornton; Marion, 2004)

$$c^2 \frac{\partial^2 u}{\partial x^2} = \frac{\partial^2 u}{\partial t^2}; \quad (5.7)$$

$$g(x, u, t) = c^2 \frac{\partial^2 u}{\partial x^2} - \frac{\partial^2 u}{\partial t^2} = 0, \quad (5.8)$$

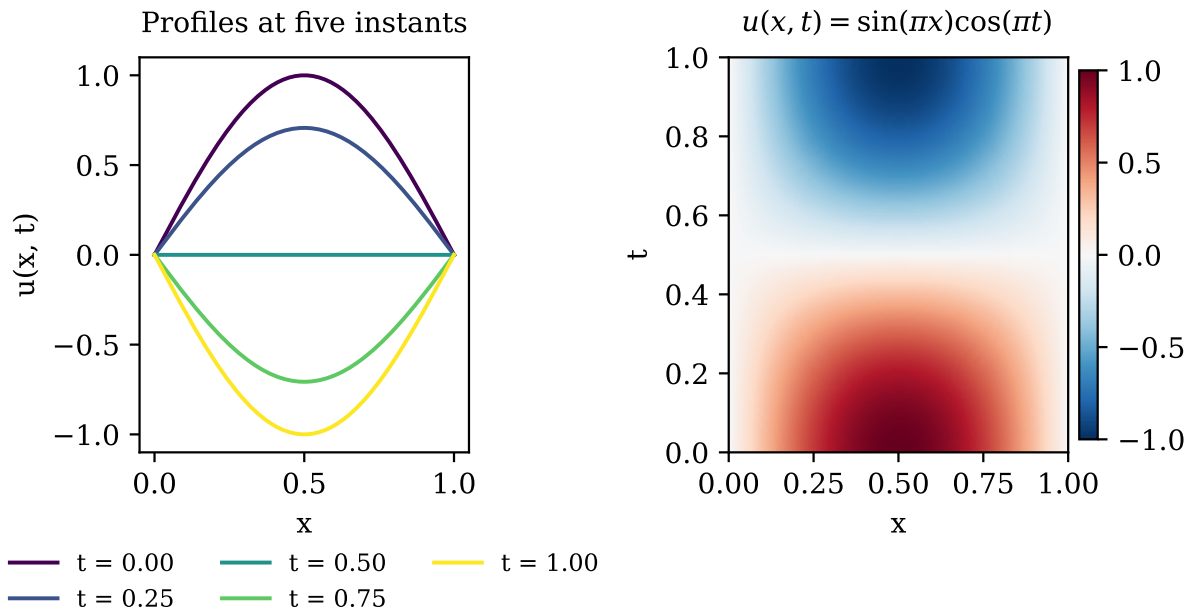
where u is the displacement of the string at position x and time t , and c is the propagation speed of the wave in the medium. We used the dimensionless problem, with $x, t \in [0, 1]$ and $c = 1$, initial displacement $u(x, 0) = \sin(\pi x)$, zero initial velocity,

$\partial u / \partial t(x, 0) = 0$, and fixed ends, $u(0, t) = u(1, t) = 0$. For these conditions, the exact solution is the standing wave

$$u(x, t) = \sin(\pi x) \cos(\pi t), \quad (5.9)$$

which performs half an oscillation period in the interval, starting from the maximum displacement at $t = 0$, passing through rest at $t = 0.5$, and reaching the opposite displacement at $t = 1$, as shown in Figure 7.

Figure 7 – Exact solution of the vibrating string problem used in this work: displacement profiles at five instants and map of $u(x, t)$.



Source: Prepared by the author.

The network, the collocation grid, the optimizer, and the learning rate are the same as in the heat transfer problem. The loss function adds a fifth term for the zero initial velocity condition, computed by automatic differentiation at the 100 points at $t = 0$.

5.6 Neuroevolution Results

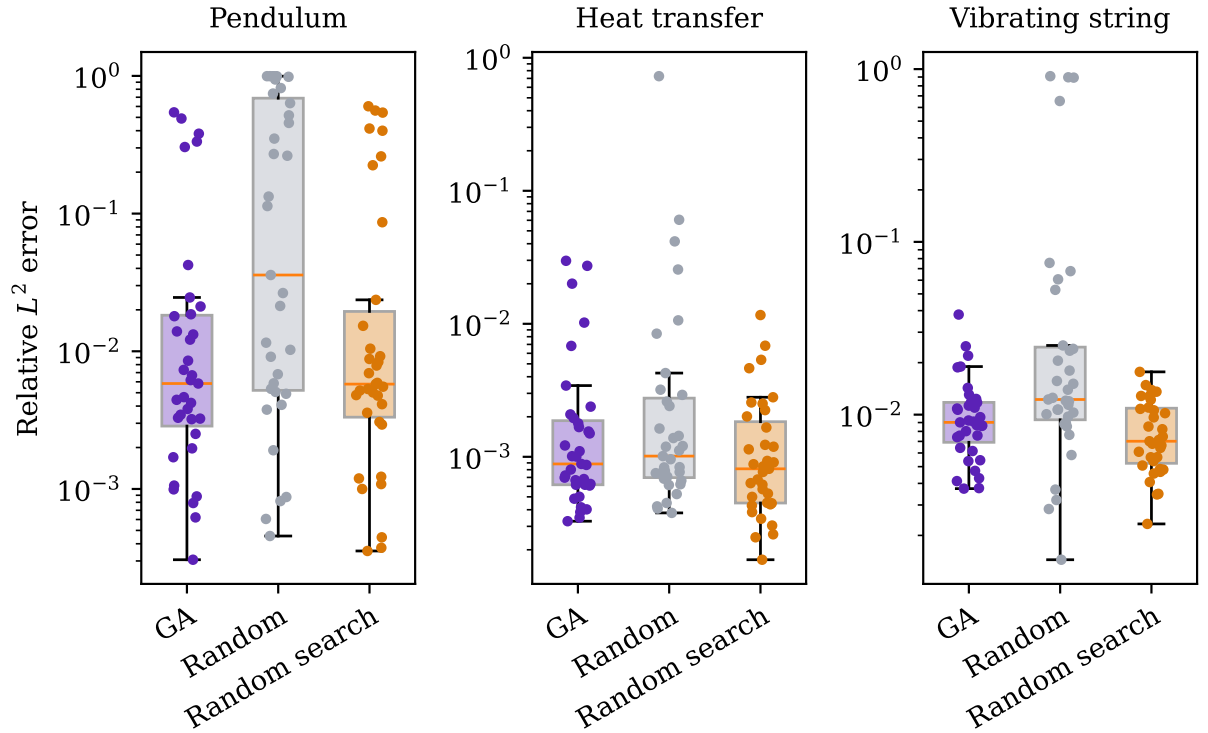
All 105 runs (35 per problem) were completed, and no training diverged. Table 4 gives the median relative L^2 error of the final network of each arm, and Figure 8 shows the distribution of this error over the 35 runs of each problem. The three arms obtained accurate solutions in most runs: Figure 9 shows the PINN obtained with the GA architecture in the median-error run of each problem, whose maximum absolute error is 0.004 for heat transfer and 0.018 for the vibrating string, for a solution of unit amplitude.

Table 4 – Median relative L^2 error and median number of trainable parameters of the final network of each arm (35 runs per problem), and Spearman correlation ρ_S between the *fitness* and the relative L^2 error of the 350 candidates screened by the GA in each problem.

Problem	Median relative L^2 error			Median parameters			ρ_S
	GA	Random	Random search	GA	Random	Random search	
Pendulum	5.8×10^{-3}	0.036	5.8×10^{-3}	1,966	505	1,726	0.79
Heat transfer	8.8×10^{-4}	1.0×10^{-3}	8.1×10^{-4}	1,947	517	1,217	0.93
Vibrating string	9.0×10^{-3}	0.012	7.0×10^{-3}	881	517	681	0.99

Source: Prepared by the author.

Figure 8 – Relative L^2 error of the final network of each arm in the 35 runs of each problem, on a logarithmic scale. The boxes show the quartiles, and each point is one run.

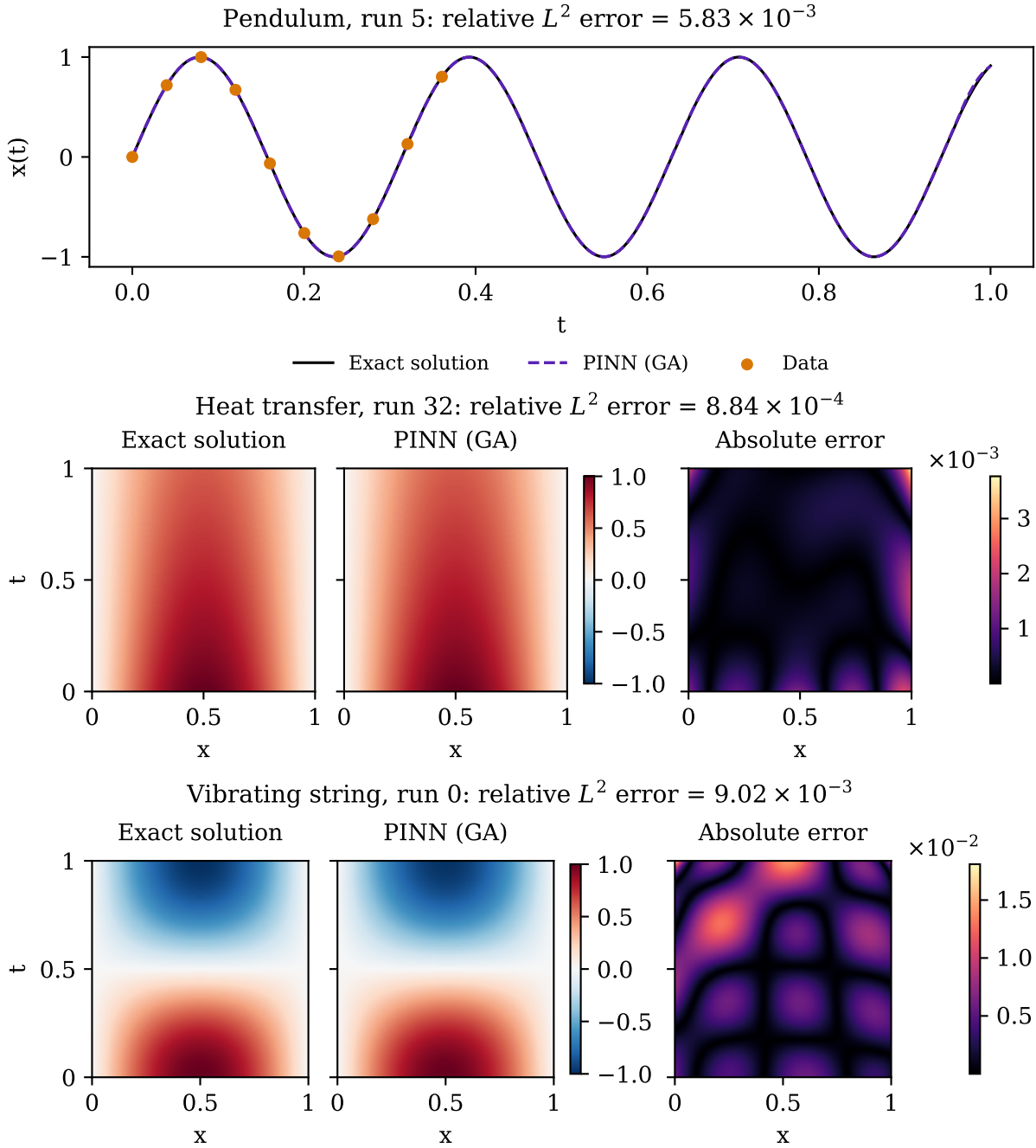


Source: Prepared by the author.

The main difference between the arms lies in the failures, i.e., the runs in which the PINN did not reproduce the solution (relative error above 0.5): the random architecture failed in 11 pendulum runs, in one heat transfer run, and in four vibrating string runs, whereas the GA failed in a single pendulum run and random search in three, also in the pendulum. The pendulum is the most demanding problem, because the PINN must extrapolate the oscillation far beyond the observed interval, and networks with few parameters cannot do so within 20,000 epochs: of the 11 random architectures that failed in the pendulum, 10 had at most 106 trainable parameters, such as (1, 2), (3, 1), and (4,

5), against a median of 613 parameters among those that did not fail.

Figure 9 – PINN obtained with the GA architecture in the median-error run of each problem. Top: pendulum, with the ten observations. Middle and bottom: exact solution, PINN prediction, and absolute error for heat transfer and for the vibrating string.



Source: Prepared by the author.

5.6.1 GA versus Random Architecture

Table 5 presents the paired comparisons between the GA and each reference. Against the random architecture, the GA obtained a lower error in 23, 21, and 26 of the 35 runs of the pendulum, heat transfer, and vibrating string, respectively, and the median ratio between the error of the random architecture and that of the GA was 4.6, 1.5, and 1.8. After the Holm correction, the improvement is statistically significant for the vibrating string under both tests ($p_B = 0.018$ and $p_W = 0.003$) and for the pendulum under the Wilcoxon test ($p_W = 0.027$).

Table 5 – Paired comparisons of the GA with each reference over the 35 runs of each problem, on the relative L^2 error. Wins: runs in which the GA obtained the lower error, with the Wilson 95% interval of the win rate; p_B : one-sided binomial test; p_W : one-sided Wilcoxon signed-rank test; both Holm-adjusted over the six comparisons. Median ratio: median of the reference error divided by the GA error, with a bootstrap 95% interval (values above 1 favor the GA). Wins (loss): wins on the secondary metric.

Problem	Reference	Wins	Wilson 95%	p_B	p_W	Median ratio [95% CI]	Wins (loss)
Pendulum	Random	23/35	49%–79%	0.224	0.027	4.62 [0.74, 29.76]	22/35
	Random search	20/35	41%–72%	0.749	0.881	1.58 [0.31, 4.02]	17/35
Heat transfer	Random	21/35	44%–74%	0.621	0.736	1.49 [0.57, 2.25]	18/35
	Random search	16/35	30%–62%	1.000	1.000	0.93 [0.65, 1.30]	10/35
Vibrating string	Random	26/35	58%–86%	0.018	3.3×10^{-3}	1.78 [1.24, 2.59]	22/35
	Random search	12/35	21%–51%	1.000	1.000	0.76 [0.64, 1.10]	11/35

Source: Prepared by the author.

In the pendulum, the binomial test alone is not significant after the correction ($p_B = 0.22$, or 0.045 before the correction): the GA loses in 12 runs, but these losses are small, whereas several of its wins are by orders of magnitude, against random architectures that failed. The 11 largest absolute differences between the errors were all wins of the GA. The Wilcoxon test, which considers the size of the paired differences, captures this asymmetry; the binomial test, which considers only the sign, does not. In heat transfer, no comparison is significant: almost every architecture with hyperbolic tangent activations solved this smooth, diffusive problem with an error of the order of 10^{-3} , so the choice of the architecture made little difference.

5.6.2 GA versus Random Search with the Same Budget

Against random search with the same budget, the GA was not superior in any of the problems. It won in 20, 16, and 12 of the 35 runs, with median ratios of 1.58, 0.93, and 0.76, and all adjusted p -values are high, from 0.75 to 1. In the vibrating string, the comparison tends to favor random search, which obtained a lower error in 23 of the 35 runs; a Wilcoxon test in the opposite direction, which was not part of the planned analysis and should be seen as exploratory, gives $p = 0.022$ before and $p = 0.066$ after the Holm

correction over the three problems, i.e., not significant. The secondary metric leads to the same conclusions: the GA obtained a lower final loss than the random architecture in 22, 18, and 22 runs, and than random search in only 17, 10, and 11 runs.

Figure 10 shows the loss along the full training of the three arms in the median-error GA run of each problem. The loss oscillates strongly under Adam, with short spikes of two orders of magnitude or more, which justifies taking the *fitness* as a mean over the last epochs. In the pendulum run, the random architecture, with 7 hidden layers of 7 neurons, converges much more slowly than the two architectures obtained by search, whereas in the heat transfer run the three networks end with similar losses and errors.

5.6.3 Partial Training as a Proxy

The method relies on the hypothesis that the loss after a short training ranks the candidates as the error would. Over the 350 candidates screened by the GA in each problem (10 per run), the Spearman correlation between the *fitness* after E_{fit} epochs and the relative L^2 error at the same point of the training was 0.79 for the pendulum, 0.93 for heat transfer, and 0.99 for the vibrating string (Table 4), all with $p < 10^{-75}$. The lower value for the pendulum is consistent with Table 2: after 1,000 epochs, the candidates are still on the initial plateau of training, and their *fitness* values are very close. Among the 105 final networks of each problem, the correlation between the final loss and the error was 0.91, 0.51, and 0.67, respectively. Therefore, the loss of the PINN is a reliable criterion for ranking the candidates in this setting, without resorting to the exact solution, although it becomes less informative among networks that are all accurate.

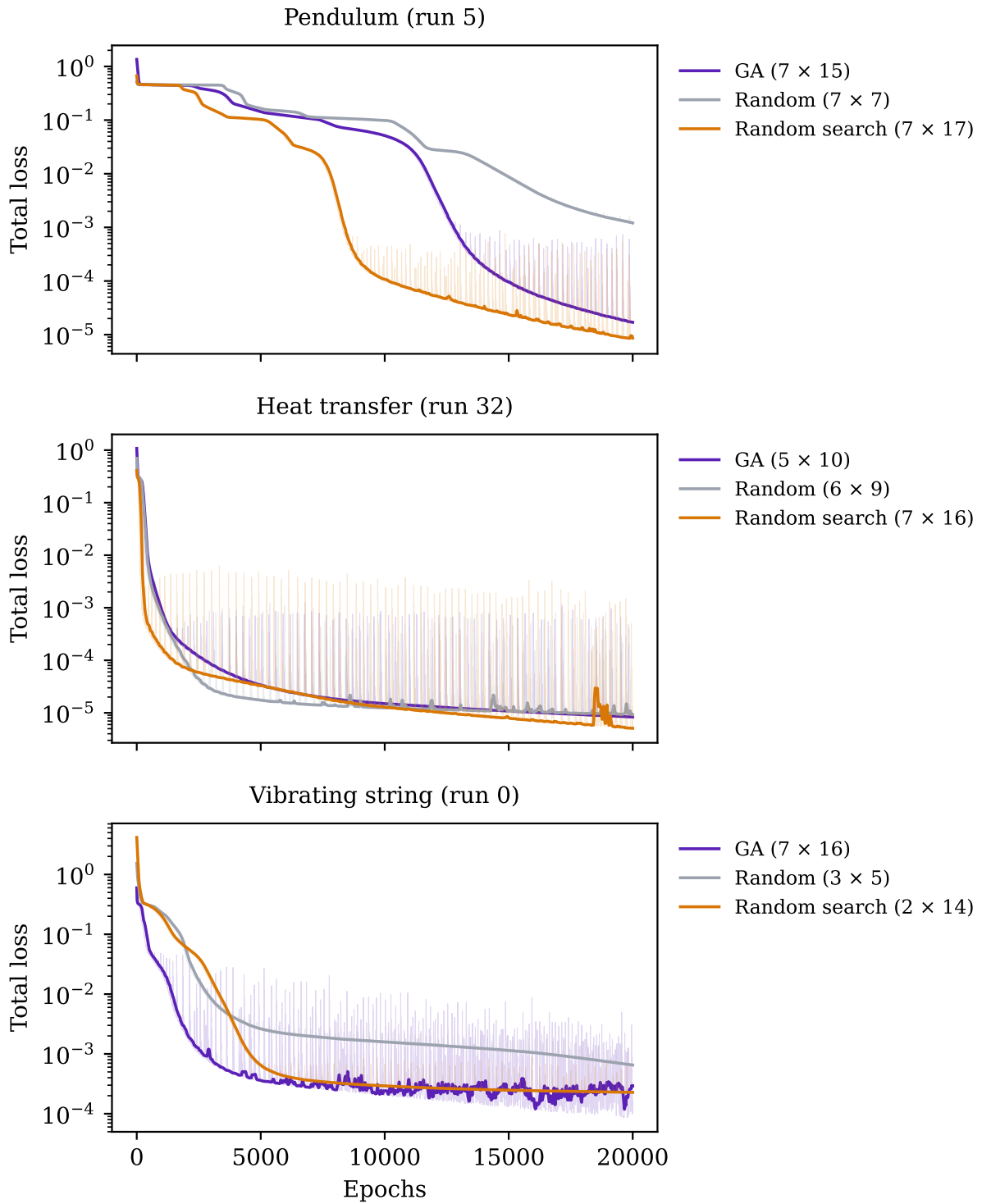
5.6.4 Selected Architectures and Computational Cost

The architectures chosen by the GA were the largest of the three arms, with medians of 1,966, 1,947, and 881 trainable parameters, against 1,726, 1,217, and 681 for random search (Table 4), as expected from a mutation that only adds layers and neurons. In the vibrating string, both searches preferred shallow and wide networks, with medians of 4 and 3 hidden layers of 16 neurons, whereas in the other problems they chose deeper networks, with 7 to 9 hidden layers. Since the GA trains larger networks in the re-evaluation, its search took 25% to 35% more time than random search with the same number of epochs, with medians of 177, 733, and 1,078 seconds per run, against 142, 546, and 808 seconds. The 105 runs took about 23 hours in total, with three runs in parallel on the GPU: 2.4 hours for the pendulum, 9.2 hours for heat transfer, and 11.6 hours for the vibrating string.

5.7 Discussion

The results confirm part of the initial hypothesis of this work. A search guided by partial trainings, costing about 1.5 full trainings, chooses PINN architectures that are

Figure 10 – Total loss along the full training of the three arms in the median-error GA run of each problem. The thin lines show the loss at every epoch, and the thick lines its rolling median over 200 epochs. The legends give the architectures as hidden layers $\times$ neurons per layer.



Source: Prepared by the author.

more accurate and, above all, more reliable than a random choice: the GA avoided almost all failures of the random architecture, and, in a typical run, the error of the random architecture was 1.5 to 4.6 times that of the GA (median ratio), with statistical significance for the pendulum and the vibrating string.

The evolutionary operators, however, did not add value over random search with the same budget. A plausible explanation is how the budget is allocated. Random search ranks 10 independent candidates with 3,000 epochs each, whereas the GA ranks them with only 1,000 epochs and spends the rest of the budget re-evaluating the five best and five offspring that differ from them by at most one layer and two neurons per layer. With a proxy as reliable as the one measured in Section 5.6.3, a longer evaluation of diverse candidates seems to be worth more than small local variations of the best ones. This result agrees with the neural architecture search literature, in which random search is a strong baseline that is often hard to beat (Li; Talwalkar, 2020), and contrasts with that of PINN/GA (Kaplarević-Mališić *et al.*, 2023), which outperforms random search using many generations, crossover, and a much larger computational budget.

Some limitations must be considered. First, a single generation is executed, and mutation only enlarges the network, which biases the search toward larger architectures and limits its exploration; a GA with several generations and bidirectional mutation could behave differently. Second, the search space has only two genes, with uniform width, and the learning rate and the other hyperparameters were fixed. Third, the three problems are linear, low-dimensional, and have smooth solutions; in heat transfer, in particular, the architecture barely matters, and harder problems are needed to discriminate between search strategies. Finally, the GPU runs are reproducible on the same *hardware* and with the same *software* versions, which are recorded with each run, but may differ slightly on other platforms.

6 CONCLUSIONS

This work aimed to introduce the concept of neuroevolution into the definition of the architecture of *Physics Informed Neural Networks* (PINNs), seeking to reduce the computational cost associated with the manual or exhaustive choice of the neural network architecture, discussed in the introduction of this work through the example of Llama 3 (Grattafiori *et al.*, 2024). To this end, three classical physical problems with exact solutions, described by ordinary and partial differential equations, were studied: the simple pendulum, heat transfer in a one-dimensional bar, and the vibrating string.

First, the performance of a standard neural network, trained only with the MSE loss, was compared with that of a PINN, also trained with a physics loss based on the differential equation of the pendulum. The results showed that, although the PINN needs more epochs than the standard network to interpolate the training data, only the PINN was able to generalize and correctly extrapolate the behavior of the physical system beyond the interval of observed data, with a relative L^2 error of 0.003 against 5.6 for the standard network, which confirms the need to incorporate physics into the training process in scenarios with few initial data.

Next, neuroevolution was applied, through a single-generation Genetic Algorithm that evolves the number of hidden layers and of neurons per layer, to select the architecture of the PINN in each of the three problems, and the result was compared with a randomly chosen architecture and with a random search with the same epoch budget. In 35 reproducible runs per problem, the architecture selected by the GA obtained a lower relative L^2 error than the random architecture in 23, 21, and 26 runs, with a median ratio of 1.5 to 4.6 between the error of the random architecture and that of the GA, and avoided almost all failures of the random choice. The improvement was statistically significant for the pendulum and the vibrating string, but not for heat transfer, a smooth problem that almost any architecture solved accurately. These results indicate that the search process, even using only a fraction of the total training epochs to evaluate each candidate architecture, identifies architectures better than a random choice.

The GA, however, did not outperform random search with the same budget in any of the problems. This indicates that, with a single generation, the gain comes from evaluating several candidates with partial trainings, and not from the evolutionary operators themselves. This conclusion is supported by the finding that the loss after partial training is a reliable proxy for the error of the candidates, with Spearman correlations from 0.79 to 0.99: with such a proxy, evaluating diverse candidates for longer, as random search does, proved as effective as locally refining the best ones, as the GA does.

Some limitations of this work should be highlighted. The three physical problems addressed are relatively simple, low-dimensional, and described by linear and well-established differential equations; the behavior of the proposed method on problems of greater complexity, dimensionality, or nonlinearity was therefore not evaluated. Moreover, the search space of the neuroevolution was restricted to the number of hidden layers and of neurons per layer of an MLP with uniform width, mutation only enlarges the network, and the algorithm executes a single generation. Finally, this revised version corrects the heat transfer and vibrating string experiments of the original version, whose networks used the ReLU activation, which zeroes the second derivatives present in these equations; all experiments were rerun with the hyperbolic tangent, exact solutions, and fixed seeds, and the code and results are publicly available.

As future work, we suggest, first, comparing the GA with random search and with other metaheuristics and optimization algorithms, such as *Particle Swarm Optimization* (PSO), Evolution Strategies (such as CMA-ES), and Bayesian Optimization, under the same computational budget, on harder and nonlinear problems, such as the Burgers and Navier–Stokes equations, in which the choice of the architecture tends to matter more. We also suggest studying the allocation of the budget between the number of candidates and the length of their evaluation, and executing several generations with bidirectional mutation and crossover. Finally, we suggest extending the search space beyond the number of layers and neurons, including different widths across layers, activation functions, the learning rate, and regularization techniques, and exploring other types of architecture besides the MLP, such as Convolutional Neural Networks (CNN), recurrent networks of the *Long Short-Term Memory* (LSTM) and *Gated Recurrent Unit* (GRU) types, and *Transformer*-based architectures.

REFERENCES

- AKUT, R.; KULKARNI, S. Neuroevolution: Using genetic algorithm for optimal design of deep learning models. *In: IEEE. 2019 IEEE International Conference on Electrical, Computer and Communication Technologies (ICECCT)*. [*S.l.: s.n.*], 2019. p. 1–6.
- BRIGATO, L.; IOCCHI, L. A close look at deep learning with small data. *In: IEEE. 2020 25th international conference on pattern recognition (ICPR)*. [*S.l.: s.n.*], 2021. p. 2490–2497.
- CUOMO, S. *et al.* Scientific machine learning through physics-informed neural networks: Where we are and what’s next. **Journal of Scientific Computing**, v. 92, n. 3, p. 88, 2022.
- EFRON, B. Bootstrap methods: another look at the jackknife. **The Annals of Statistics**, v. 7, n. 1, p. 1–26, 1979.
- EIBEN, A. E.; SMITH, J. E. **Introduction to Evolutionary Computing**. 2. ed. [*S.l.: s.n.*]: Springer, 2015.
- GÉRON, A. **Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow**. 2. ed. Sebastopol: O’Reilly Media, 2019.
- GOLDBERG, D. E. **Genetic Algorithms in Search, Optimization, and Machine Learning**. [*S.l.: s.n.*]: Addison-Wesley, 1989.
- GRATTAFIORI, A. *et al.* The Llama 3 herd of models. **arXiv preprint arXiv:2407.21783**, 2024.
- HAHN, D. W.; ÖZISIK, M. N. **Heat conduction**. [*S.l.: s.n.*]: John Wiley & Sons, 2012.
- HOLLAND, J. H. Genetic algorithms. **Scientific American**, v. 267, n. 1, p. 66–73, 1992.
- HOLM, S. A simple sequentially rejective multiple test procedure. **Scandinavian Journal of Statistics**, v. 6, n. 2, p. 65–70, 1979.
- KAPLAREVIĆ-MALIŠIĆ, A. *et al.* Identifying optimal architectures of physics-informed neural networks by evolutionary strategy. **Applied Soft Computing**, v. 146, p. 110646, 2023.
- KARNIADAKIS, G. E. *et al.* Physics-informed machine learning. **Nature Reviews Physics**, v. 3, n. 6, p. 422–440, 2021.
- KINGMA, D. P.; BA, J. Adam: A method for stochastic optimization. **arXiv preprint arXiv:1412.6980**, 2014.
- LI, L.; TALWALKAR, A. Random search and reproducibility for neural architecture search. *In: Proceedings of the 35th Uncertainty in Artificial Intelligence Conference*. [*S.l.: s.n.*]: PMLR, 2020. (Proceedings of Machine Learning Research, v. 115), p. 367–377.

RAISSI, M.; PERDIKARIS, P.; KARNIADAKIS, G. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. **Journal of Computational Physics**, v. 378, p. 686–707, 2019. ISSN 0021-9991. Available at: <https://www.sciencedirect.com/science/article/pii/S0021999118307125>.

SPEARMAN, C. The proof and measurement of association between two things. **The American Journal of Psychology**, v. 15, n. 1, p. 72–101, 1904.

STANLEY, K. O. *et al.* Designing neural networks through neuroevolution. **Nature Machine Intelligence**, Nature Publishing Group UK London, v. 1, n. 1, p. 24–35, 2019.

STANLEY, K. O.; MIIKKULAINEN, R. Evolving neural networks through augmenting topologies. **Evolutionary Computation**, MIT Press, v. 10, n. 2, p. 99–127, 2002.

SUNG, N. W. Y. *et al.* Neuroevolution of physics-informed neural nets: Benchmark problems and comparative results. *In: Proceedings of the Companion Conference on Genetic and Evolutionary Computation (GECCO)*. [S.l.: s.n.], 2023.

THORNTON, S. T.; MARION, J. **Classical dynamics of particles and systems**. 5. ed. Belmont: Thomson Brooks/Cole, 2004. 672 p.

WIDROW, B.; HOFF, M. E. Adaptive switching circuits. *In: IRE WESCON Convention Record*. [S.l.: s.n.], 1960. v. 4, p. 96–104.

WILCOXON, F. Individual comparisons by ranking methods. **Biometrics Bulletin**, v. 1, n. 6, p. 80–83, 1945.

WILSON, E. B. Probable inference, the law of succession, and statistical inference. **Journal of the American Statistical Association**, v. 22, n. 158, p. 209–212, 1927.

WONG, J. C. *et al.* Evolutionary optimization of physics-informed neural networks: Evo-PINN frontiers and opportunities. **IEEE Computational Intelligence Magazine**, 2026. ArXiv:2501.06572.

YAO, X. Evolving artificial neural networks. **Proceedings of the IEEE**, IEEE, v. 87, n. 9, p. 1423–1447, 1999.